A Five-Layer Diagnostic Stack for Multi-Agent LLM Systems: Collaboration, Architecture, Evaluation, Security, and Communication

Anonymous Author(s)

Multi-agent LLM collaboration is organized here as a five-layer diagnostic stack—failure diagnosis and suppression, collaboration architecture and role design, evaluation benchmark and metric measurement, adversarial safety and federated constraint protection, and natural language and communication coordination—in which bottom-up constraint propagation and top-down feedback calibration link the layers. We argue that failures, architectures, evaluations, safety, and communication constraints are systematically decoupled, so that each layer's simplifications propagate as unmeasured risks to the others. This decoupling is hard to see because the literature reports named failure modes, architecture taxonomies, benchmarks, threat models, and coordination studies in separate venues with incompatible vocabularies, leaving cross-layer dependencies invisible. We reconstruct the stack by systematically re-organizing the evidence: a 30-mode inventory of named failures, with behavioral drift, constraint drift, and cascades forming the main trunk, is traced to interface-level vulnerabilities that recur across failure modes, architectures are placed in a 2×2 design space defined by orchestration topology and role regime, whose two axes interact rather than vary independently, benchmarks are mapped against capability coverage, adversarial threats are grouped into five named surfaces with explicit attacker capability requirements, and communication effects are attributed to ambiguity, context limits, and context unavailability. Three positioning choices organize this review: it treats drift and cascading hallucination as related manifestations of agent-to-agent interfaces while keeping their mechanisms distinct; it replaces overlapping architecture taxonomies with two principal design dimensions that place all reported architecture designs in one comparable space along the topology–role plane, while security remains a cross-cutting constraint rather than a third design axis; and it reads benchmark saturation and safety threat models as evidence of the same measurement gap. The resulting stack reframes decoupling as the field's central risk and supplies a common vocabulary plus three comparable design coordinates (Tables 3, 4, and 5).

Additional Key Words and Phrases: multi-agent collaboration, large language models, agent orchestration, evaluation benchmarks, agent security

1 INTRODUCTION

Large language model (LLM) agents now routinely delegate to one another: a planner spawns researchers, a reviewer critiques a coder, a supervisor routes subtasks across a swarm. This shift from single-agent prompting to multi-agent collaboration has produced a fast-growing literature on orchestration frameworks, role assignment schemes, and communication protocols, exemplified by MetaGPT's meta-programming of standardized agent roles and LLM-Collab's chain-of-thought task decomposition. Surveys have mapped the space of agentic reasoning and autonomous agents broadly, and several have catalogued multi-agent techniques and their business implications. Yet the field's central problem is not a shortage of architectures but a shortage of understanding about what goes wrong between them. Errors introduced by one agent do not stay local: they are re-encoded in messages, inherited by downstream roles, and amplified across turns. Agent drift—the progressive degradation of decision quality and inter-agent consistency—has been reported to substantially reduce task success and increase inter-agent conflict over long interaction sequences. Communication-channel attacks such as agent-in-the-middle interception have been reported to achieve high success rates in many configurations without altering agent profiles, and prompt-layer impersonation has been reported to reach high success rates in swarm settings. These findings share a structural cause: information, intent, and constraints decay or distort at the interfaces of delegation, communication, memory, and optimization.

The literature has responded layer by layer, but each response has been developed in relative isolation. Failure-mode research names 30 distinct modes, which we group into six families, with behavioral drift,

constraint drift, and cascades forming the main trunk; it also proposes suppression mechanisms whose evidence strength varies widely across studies. Architecture research offers at least six mutually incompatible classification schemes, with centralized and decentralized topologies each claimed superior under different conditions. Evaluation research relies on benchmarks that are narrow, single-agent, and static: HumanEval and MBPP are near saturation for function-level code generation, and the early benchmarks catalogued in comprehensive agent surveys are uniformly flagged as non-agentic. Safety research formalizes attacker models and threat surfaces, but protection mechanisms remain only partially composable at the orchestration level. Communication research shows that ambiguity and context-window limits degrade coordination, yet explicit modeling of communication constraints as a first-class design objective remains rare: a small number of reviewed systems model delay or bandwidth directly, while most absorb these limits into generic efficiency goals. The consequence is a field in which results are stratified and mutually incomparable, and in which each layer's engineering simplification is paid for by unmeasured risk in another.

Existing surveys do not resolve this. They organize by technique family or by application domain, treating architectures, benchmarks, and safety as separate chapters rather than as coupled layers. None provides a unified framework in which failure diagnosis constrains architecture design, architecture design determines what evaluation must measure, evaluation exposes what safety must protect, and communication engineering sets the coordination bandwidth within which all of the above operate. This survey addresses that gap.

Originality. We organize the field as a five-layer governance stack: failure diagnosis and suppression at the bottom, collaboration architecture and role design above it, evaluation benchmarks and metrics as the measurement layer, adversarial safety and federated constraints as the protection layer, and natural language and communication engineering as the coordination layer. Constraint propagation runs bottom-up—failure modes constrain which architectures are admissible—and feedback calibration runs top-down, as evaluation evidence recalibrates architectural and safety choices. Three positioning choices organize this framing. First, it is failure-anchored: the stack is derived from named failure modes rather than from technique taxonomies, so every layer is accountable to a diagnosed pathology. Second, it is dimensionally parsimonious: we reconstruct architecture classifications into two principal dimensions—orchestration topology (centralized/decentralized) and role regime (fixed/evolving)—which place all architectures discussed in our evidence set in one comparable space and expose the naming overlaps (Star $\approx$ Hierarchical $\approx$ Orchestrator-Workers) that make prior taxonomies non-MECE. Third, it is evidence-stratified: we grade suppression mechanisms by the strength of their supporting evidence and map benchmark coverage against capability blind spots, rather than reporting mechanisms and benchmarks as a flat inventory.

Contributions. Our analysis rests on a frozen evidence matrix spanning five sub-question clusters, a structured evidence base of metric and limitation records, and quantitative stratification of six suppression-mechanism papers. We contribute the following falsifiable claims.

- We show that the field's named failure modes can be grouped into six families sharing interface-level vulnerabilities, with behavioral drift, constraint drift, and cascades forming the main trunk, and that drift is a major contributor to task-success loss and increased human intervention, establishing failure diagnosis as a constraint on architecture rather than a post-hoc concern [1, 2, 3]. - We show that centralized and decentralized architectures form a conditional Pareto frontier rather than a ranking: no single topology dominates on task accuracy, scalability, and traceability simultaneously—robustness depends on representation sharing rather than topology alone—and the six existing classification schemes can be replaced by a two-dimensional taxonomy that places all architectures in our evidence set along the topology–role plane, with security treated as a cross-cutting constraint. - We show that mainstream benchmarks exhibit systematic capability blind spots—most early benchmarks in comprehensive agent surveys are non-agentic, and code-generation benchmarks are near saturation—so that reported gains across studies are stratified and mutually incomparable without a shared evaluation protocol [4, 5]. - We show that safety protections are only partially composable at the orchestration level: prompt-layer attacks have been reported to reach high success rates while colluding-agent attacks have been reported to succeed in only a small fraction of cases, and interception attacks have been reported to exceed 40% success without modifying agent capabilities, indicating that protection must be specified per layer rather than per system. - We show that communication constraints are rarely modeled as a first-class design objective—only a few reviewed systems do so—and that ambiguity and context-window limits degrade coordination through intent deviation and lost-in-the-middle information loss, identifying communication engineering as the stack's least-instrumented layer.

The implication for readers is direct. Seeking a single optimal architecture or a unified metric is a category error; multi-agent LLM collaboration should be treated as a governance stack with inter-

layer coupling. Investment should prioritize explicit modeling of cross-layer constraints and comparable evaluation protocols—otherwise each layer's local simplification will continue to purchase efficiency with unmeasured risk elsewhere in the stack.

2 BACKGROUND AND PRELIMINARIES

This section establishes the conceptual vocabulary used throughout the review. We define five interlocking notions—multi-agent collaboration, orchestration topology, role regimes, evaluation benchmarks, and collaboration constraints—and clarify how they relate to one another. The definitions are deliberately operational: each concept is tied to observable design choices or measurable properties, so that later thematic sections can compare systems without ambiguity.

To translate the concern over inter-layer coupling in the governance stack into actionable design and evaluation, a unified conceptual vocabulary is first needed; the following defines five interlocking basic concepts and explains how they are related.

These communication engineering constraints, which are excluded by default in collaboration models, precisely need to be reintroduced into the analysis through a precise set of conceptual vocabulary; the following definitions are established exactly to bridge this gap, anchoring collaboration, topology, role regimes, evaluation benchmarks, and constraint conditions one by one onto observable design choices and measurable properties.

Such an instrument presupposes a shared vocabulary precise enough to compare designs across layers, so we first establish the five interlocking notions that anchor the review's analysis.

2.1 Multi-Agent Collaboration

We define multi-agent collaboration as a setting in which two or more autonomous LLM-based agents pursue a shared task through explicit interaction, division of labor, and information exchange, such that the collective outcome depends on inter-agent dependencies rather than on any single agent's output. This definition separates collaboration from mere aggregation: agents must exchange messages, share intermediate artifacts, or coordinate decisions. Early formulations demonstrated that assigning distinct subtasks to specialized agents improves design ideation and, in a separate line of work, code generation over monolithic prompting. The concept has since expanded across domains—clinical triage, electronic health records, recommendation, root-cause analysis, and hardware design—which indicates that collaboration is a general coordination paradigm rather than a task-specific trick.

Two properties recur across these instantiations. First, collaboration is structured: agents operate under some protocol that governs who speaks, when, and with what information. Second, collaboration is fallible: errors introduced by one agent can be adopted, amplified, or silently propagated by others. This failure mode is now documented as a first-class concern, with benchmarks explicitly designed to measure adversarial and cascading risks in multi-agent LLM systems. We therefore treat error propagation not as an implementation detail but as a defining property of the collaboration concept itself.

2.2 Orchestration Topology

Orchestration topology denotes the structural arrangement of control and communication among agents: who issues task assignments, whether a central coordinator exists, and how agents are connected. We distinguish three canonical topologies. Centralized topologies route planning and delegation through a single orchestrator or a fixed standard operating procedure (SOP); representative systems encode domain procedures as explicit workflows. Decentralized topologies allow agents to self-organize, negotiate, or evolve coordination structures without a persistent controller. Hybrid topologies combine a lightweight coordinator with peer-to-peer sub-structures, often realized as task graphs or configurable pipelines.

The topology choice carries direct consequences for scalability and robustness. Centralized SOPs yield predictable, auditable execution but concentrate failure at the orchestrator and scale poorly when task graphs grow. Decentralized and self-organizing designs improve adaptability and parallel throughput but complicate verification and can amplify communication overhead. Dynamic agent elimination and auto-scaling mechanisms have been proposed to recover efficiency by pruning redundant participants at runtime (e.g., for centralized settings; for decentralized settings); evidence that a single mechanism operates effectively under both topologies remains limited.

2.3 Role Regimes

A role regime specifies how agents are assigned responsibilities. We distinguish predefined fixed roles, where each agent receives a static persona or function (e.g., planner, coder, reviewer), from dynamic evolving roles, where responsibilities are reassigned or discovered during execution. Fixed-role designs underpin role-based collaboration frameworks and have been shown to improve heuristic design and, in separate work, stance detection through explicit division of labor. Structured role assignment has also been reported to yield measurable stability gains when roles are derived from structural information principles; however, the specific stability metrics examined in that study should be interpreted with care. Dynamic regimes instead let agents adopt, swap, or retire roles in response to task progress, as in reward-driven self-organization and organized-team cooperation studies.

Role regimes interact with both topology and error propagation. Fixed roles reduce ambiguity about responsibility and thus make it easier to localize an error to a specific agent, but they are brittle when the task distribution shifts. Dynamic roles increase flexibility and can absorb local failures through reassignment, yet they introduce coordination ambiguity that itself becomes a propagation channel. Role design is therefore not a cosmetic choice: it determines the granularity at which errors can be attributed and contained.

2.4 Evaluation Benchmarks and Metrics

Evaluation benchmarks are standardized task collections with associated metrics that permit comparison across systems. The benchmarks most frequently reused in multi-agent LLM work span code generation (HumanEval, MBPP), general knowledge and reasoning (MMLU, GSM8K, MATH), and database querying (Spider, BIRD). Their coverage differs sharply: code benchmarks emphasize functional correctness under unit tests, mathematical benchmarks emphasize final-answer accuracy, and text-to-SQL benchmarks emphasize execution correctness against a schema. Multi-agent systems have been evaluated on these suites both to demonstrate task gains and to expose coordination failures.

Metrics inherit these emphases. Accuracy and Pass@1 dominate reasoning and code settings; success rate is common in agentic and embodied tasks; completeness and executability appear in generation and design pipelines where partial outputs are meaningful. Because these metrics were designed for single-model evaluation, they do not capture collaboration-specific properties such as error tolerance, coordination cost, or robustness to a single agent's failure. Recent work has begun to fill this gap with dedicated adversarial and fairness benchmarks, but no unified framework yet spans task type, collaboration scale, error tolerance, and cost. We adopt this gap as an organizing motivation for the review.

2.5 Collaboration Constraints

Collaboration constraints are the resource and information limits under which agents coordinate. We group them into three families. Communication constraints include latency, bandwidth, message size, and the availability of dedicated control channels; delay-tolerant and communication-efficient designs explicitly model these limits. Context constraints include context-window size and the amount of shared history an agent can retain, which motivate shared-memory and structured-handoff mechanisms. Security and privacy constraints include adversarial manipulation of inter-agent messages and the bounded-information assumptions of federated or client-aided protocols.

These constraints are not independent of the earlier concepts. Tight communication budgets favor centralized, SOP-like topologies with short structured handoffs, whereas loose budgets permit decentralized, conversational coordination. Context limits determine how much shared state can anchor agent behavior, and thus how strongly errors can be detected before propagation. Security constraints determine whether messages can be trusted at all, which in turn shapes the protection mechanisms a system can compose. We therefore treat constraints as the boundary conditions under which topology, roles, and evaluation choices must be jointly optimized, and we use this framing to organize the thematic chapters that follow.

3 RELATED WORK

Large language model (LLM) driven multi-agent collaboration has moved from proof-of-concept demonstrations to a crowded research front, and the secondary literature has begun to consolidate around a recognizable set of organizing perspectives. The earliest survey-style treatments framed the problem through the lens of coordination and communication inherited from classical multi-agent systems, treating LLM agents as a new substrate for long-standing questions about how autonomous entities share state and

negotiate joint action. A second wave reframed the field around agent workflows, cataloguing the control-flow patterns by which planners, executors, and reviewers are stitched together [6], while a parallel strand surveyed the broader trajectory from LLM reasoning to autonomous agents and, in doing so, assembled the first comparative inventory of benchmarks and frameworks. More recent overviews have shifted toward architecture and business framing, offering taxonomies of coordination forms and deployment patterns and enumerating the open problems that arise when LLM-based systems are scaled into multi-agent configurations [7]. Across these surveys the conclusions converge on three claims: that orchestration choices dominate downstream performance, that evaluation remains fragmented, and that failure modes are acknowledged but rarely systematized. This chapter takes those claims as its starting point and organizes the primary literature into the methodological, evaluative, application-oriented, and adversarial streams that the surveys describe, before identifying where the present review departs from them.

The first and largest stream treats collaboration as an orchestration and control-flow design problem. Work in this tradition asks how to decompose a task, assign sub-goals to agents, and sequence their interactions so that the aggregate system outperforms a single model. Representative systems adopt role-specialized pipelines in which distinct agents assume planner, coder, and reviewer responsibilities [8], hierarchical task allocation with integrated memory to keep long-horizon embodied agents coherent [9], and dialectic multi-robot collaboration in which LLM agents debate candidate plans before execution [10]. Related designs scale the same idea to heterogeneous embodied teams [11] and to robotic autonomy more broadly [12], while comparative work asks whether centralized or decentralized control is preferable as the number of robots grows. At the enterprise and infrastructure level, blueprint architectures for compound AI systems describe how agents and data sources are orchestrated together [13], and reference architectures for foundation-model agents translate responsible-AI requirements into concrete orchestration components. A recurring theme across this stream is that the orchestration graph itself, rather than any individual agent, is the object of design; the present review shares this premise but treats orchestration as one axis of a broader taxonomy rather than as the organizing principle.

A closely related body of work studies role assignment and the division of labor, often borrowing from human organizational structures. MetaGPT-style approaches encode standardized operating procedures so that agents with distinct roles produce intermediate artifacts that constrain one another [14]; role-discovery methods based on structural information show that such constraints can be learned rather than hand-specified [15], and role-aligned mixture-of-experts architectures apply the same logic to domain-specific assistants such as legal consultation [16]. In software engineering, role specialization has been used to formalize requirements for reliable code generation, to estimate agile effort through collaborative review and discussion [17], and to generate and tune human-readable system instructions through feedback-driven agent loops [18]. Design-oriented work applies multi-agent role decomposition to product design workflows [19] and to educational tutoring, where cognitive evolution across roles is used to sustain learner engagement. The evidence from these studies suggests that role structure improves interpretability and traceability, a point made explicit by work on accountability in role-specialized pipelines [8], yet the literature offers little agreement on how roles should be specified, how many are needed, or when specialization degrades into redundancy. This review treats role structure as a design variable whose interaction with failure behavior remains under-examined.

Communication and information sharing form a third stream, and here the LLM literature connects most directly to classical multi-agent reinforcement learning. Surveys of communication in multi-agent deep reinforcement learning established the core vocabulary of learned, differentiable, and emergent messaging [20], and subsequent work demonstrated memory-driven communication for small-scale coordination [21], learnable handshake protocols for collaborative perception [22], and representational communication for large-scale traffic signal control. Emergent communication has been studied specifically for future wireless networks [23], and task-oriented agent communication has been reframed as the transmission of machine language tokens rather than natural language. Within LLM systems, the Model Context Protocol has emerged as a standardization layer for tool and context exchange, with work extending it to wireless environment-aware agents [24], to manufacturing capability modeling [25], and to financial tool-use benchmarking [26]. A persistent concern across both the reinforcement learning and LLM strands is that natural-language coordination is inherently ambiguous, forcing repeated remedial specification and sometimes causing execution to drift from user intent or to loop indefinitely [27]. The present review differs from this stream in treating communication constraints not as engineering parameters to be optimized but as structural conditions that shape how errors propagate.

A fourth stream addresses evaluation, and it is here that the literature is most visibly fragmented. Comprehensive reviews of autonomous agents have catalogued the proliferation of benchmarks while noting the absence of a unifying evaluation framework [4]. Individual studies anchor on established suites:

HumanEval and MBPP for function-level code generation, MMLU for academic knowledge, GSM8K and MATH for mathematical reasoning, and Spider and BIRD (with WikiSQL as an earlier precursor) for structured query generation, a grouping made explicit in recent synthesis work [4]. Domain-specific benchmarks have multiplied in parallel, including V2X-Sim for collaborative perception in autonomous driving [28], ASIC-Agent for autonomous chip design [29], FinMCP-Bench for financial tool use under the Model Context Protocol [26], and realistic benchmarks for poisoning attacks on tool-using agents. Evaluation methodology has also been extended toward specification-level assessment of multi-agent LLM systems in cybersecurity settings and toward adversarial risk benchmarking for multi-agent configurations. The consequence of this proliferation is that accuracy, success rate, pass@1, and task-specific completeness metrics are rarely comparable across studies, and the semantic relationship among them is left implicit. This review responds to that gap by treating metric alignment as a first-class concern rather than a reporting detail.

Application-driven work constitutes a fifth and rapidly expanding stream, and its breadth is itself informative about where collaboration pays off. In software and hardware engineering, multi-agent systems have been applied to RTL code generation [30], chiplet design with adaptive debugging [31], intrinsic code translation for RISC-V vector extensions [32], REST API testing with semantic graphs [33], HPC unit test generation [34], and firmware validation and patching. Security operations supply another cluster, including malicious package dissection [35], network protocol fuzzing [36], open-source intelligence gathering [37], autonomous cyber defense [38], and supply-chain vulnerability mitigation. Data-centric applications include schema-aware natural language to SQL generation [39], multimodal entity linking, tabular data analysis through conversational agents [40], animated data video creation, and feature engineering for industrial event logs [41]. Scientific and creative domains are represented by neural architecture design [42], compositional text-to-image generation [43], competitive debate [44], and empathetic response generation. What unites these otherwise disparate efforts is a shared implicit hypothesis, namely that decomposition into specialized agents mitigates single-model context limits and hallucination; the present review treats that hypothesis as an empirical claim to be tested rather than assumed.

A sixth stream situates multi-agent collaboration within networked and wireless systems, where coordination must respect bandwidth, latency, and decentralization. Tutorial treatments of deep reinforcement learning for wireless networks provide the methodological foundation [45], and multi-agent reinforcement learning has been applied to wireless MAC protocol emergence [46], generalized MAC protocol learning via abstraction [47], and traffic signal control at scale. LLM-enabled multi-agent frameworks have been proposed for 6G networks with dual-loop edge-terminal collaboration [48], for autonomous network orchestration combining LLMs with continual reinforcement learning [49], and for generative 6G simulation integrating LLM agents with ns-3 [48]. Semantic communication for AI agent networks has been addressed through distillation-enabled knowledge alignment [50], and agentic approaches have been applied to RAN security compliance. Federated analytics over decentralized data sources represents a further extension in which agentic LLMs operate without centralizing raw data [51]. This stream is notable for making communication constraints explicit, yet it remains largely disconnected from the orchestration and failure literature; the present review attempts to bridge that disconnect.

A seventh stream examines robustness, failure, and behavioral degradation, and it is the stream with which this review is most directly in dialogue. Empirical studies of autonomous agents have documented recurring failure patterns in task completion [3], and formal work has defined agent drift as the progressive degradation of behavior, decision quality, and inter-agent consistency over long interaction sequences, decomposing it into semantic, coordination, and behavioral subtypes [1]. Chaos engineering has been adapted to assess and enhance the robustness of LLM-based multi-agent systems [52], and reinforcement-learning-augmented LLM agents have been proposed to improve collaborative decision making and global performance optimization [53]. Traceability and provenance mechanisms offer a complementary angle, tracking agent interactions across workflows [54] and attributing responsibility in role-specialized pipelines [8]. Related work on planner-policy integration combines LLM-based planning with comparative policies to address coordination and safety simultaneously. Despite this activity, the literature reports failures without a shared taxonomy, and no existing synthesis connects drift, conflict, and error propagation into a single account. This review takes that integration as its central contribution.

An eighth stream concerns adversarial threats and security, where the multi-agent setting introduces attack surfaces absent from single-agent systems. Early work demonstrated adversarial attacks on multi-agent communication itself [55], and subsequent studies have examined how LLM agents empower phishing attackers and how tool-using agents can be manipulated through poisoned documentation under the Model Context Protocol. TAMAS provides a systematic formalization of attacker models for multi-agent LLM systems [56], distinguishing attacker goals, background knowledge, and injection capabilities across prompt,

agent, and environment layers. Complementary efforts benchmark adversarial risks in multi-agent configurations [56], assess robustness through controlled perturbation [52], and study language evolution as a mechanism for evading social media regulation in simulated multi-agent populations. Security-oriented applications such as malicious package dissection [35] and autonomous defense [38] show the defensive potential of the same architectures. The gap this review addresses is that adversarial work and orchestration work have developed largely in isolation, so the relationship between architecture choices and attack surface remains unmapped.

A ninth stream connects multi-agent collaboration to retrieval, generation quality, and hallucination mitigation. Retrieval-augmented generation has been extended into multi-agent pipelines that supervise the retrieval process for stability and interpretability [57], and corrective retrieval strategies have been proposed to reduce hallucination in LLM outputs [58]. Hallucination-free question answering has been pursued through automatic generation pipelines, and generative AI applications have integrated retrieval and agents into end-to-end systems. Multi-agent frameworks have also been applied to psychological test item generation [59], where role-structured generation must satisfy psychometric constraints. These efforts share the assumption that distributing generation and verification across agents improves factual reliability, but they evaluate that assumption with heterogeneous metrics and rarely report failure rates under adversarial or distribution-shifted inputs. This review treats reliability claims as requiring the same scrutiny as capability claims.

A tenth stream addresses planning, reinforcement learning, and decision-making in collaborative settings. Work on training task-reasoning LLM agents for multi-turn planning via single-turn reinforcement learning shows how planning competence can be transferred across horizons [60], and YOLO-MARL demonstrates that LLM guidance can accelerate multi-agent reinforcement learning [61]. Urgency-aware planning studies ask whether LLM agents can genuinely coordinate under time pressure [62], while CP-AgentNet uses generative agents to design communication protocols autonomously [63]. Multi-agent evolutionary systems have been applied to feature engineering [41], and reinforcement learning with LLM planners has been combined with graph policies for coordinated control. The tension across this stream is between learned policies that optimize a reward and LLM-based planners that optimize a textual objective; the two are rarely evaluated on common ground, which limits claims about which approach yields more reliable collaboration.

An eleventh stream examines human-facing and social dimensions, including simulation, debate, and education. LLM-based multi-agent simulation has been used to study language evolution under regulatory pressure, and social simulation more broadly has been surveyed as an application of LLM agents [64]. Competitive debate frameworks use multi-agent dynamics to counter hallucination and improve argumentation [44], and empathetic response generation has been structured through explicit reasoning traces [65]. Educational multi-agent systems model cognitive evolution across tutoring roles [66], and psychometric item generation applies role-structured collaboration to assessment design. These studies reveal that social and pedagogical settings impose coordination requirements, such as maintaining consistent persona and tracking learner state, that differ from those in software tasks; the present review notes this heterogeneity as a reason why a single evaluation paradigm has proven elusive.

A twelfth stream focuses on provenance, traceability, and accountability as properties that multi-agent systems must satisfy to be deployable. Unified provenance frameworks track agent interactions across agentic workflows [54], and accountability studies examine how responsibility can be attributed in role-specialized pipelines [8]. Reference architectures for responsible generative AI embed such concerns into the design of foundation-model agents, and specification and evaluation work in cybersecurity settings makes accountability an explicit evaluation target. Traceability is also relevant to failure analysis, since drift and error propagation are difficult to diagnose without interaction-level records. This review treats provenance as an enabling condition for the failure taxonomy it proposes, rather than as a separate engineering concern.

3.1 Application Domains and Deployment Evidence

The methodological streams above are complemented by an application literature that tests whether collaboration pays off under domain constraints. Data-centric deployments include semantic-accurate text-to-SQL through consistency-enhanced agents [67], multi-agent perception with distilled collaboration graphs [22], and rumour-propagation simulation at million-agent scale [64]. Hardware and systems engineering contributes chiplet design with adaptive debugging [31], configurable comparative hardware design [68], and REST API testing driven by semantic graphs [33]; security operations add network protocol fuzzing [36] and supply-chain vulnerability mitigation [69]. Scientific and professional workflows are represented by automated drug-discovery programming [70], flight-procedure design [71], multi-agent translation [72],

task-oriented dialogue consistency [73], operations-research decision support [74], and distributed online meta-learning under limited communication [75].

Enterprise deployment evidence is thinner but consistent in direction. Blueprint architectures describe how agents and data sources are orchestrated for compound AI systems [13], while industrial frameworks report task-automation gains from role-specialized agents [76] and evaluate GenAI collaboration patterns for enterprise applications [77]. Read against the methodological chapters, these deployments reinforce two findings: role specialization transfers across domains more readily than topology choice, and reported gains are rarely measured on a shared benchmark, which returns the argument to the evaluation layer.

Taken together, these streams cover orchestration, roles, communication, evaluation, applications, networked systems, robustness, security, retrieval, planning, social simulation, and provenance. The coverage is broad but uneven. Orchestration and application studies dominate, while failure behavior, adversarial risk, and evaluation semantics remain comparatively under-synthesized. Existing surveys describe architectures and enumerate benchmarks [78, 4], and classical surveys describe communication in reinforcement learning [20], but none integrates the failure literature into a coherent taxonomy, none systematically relates architecture choices to attack surface, and none reconciles the proliferating metrics into a comparable evaluation framework. Five canonical foundations frame the positioning of this review. MAST [79] derives a 14-mode failure taxonomy from annotated traces; we retain its modes as evidence but organize them by the layer at which they originate, so that a diagnosis can be traced upward through the stack rather than left as a label. MetaGPT-style SOP pipelines [14] and business-perspective taxonomies [78] classify architectures by workflow role; we replace those overlapping schemes with two principal dimensions and treat their interaction, rather than independence, as the design question. Benchmark inventories [4, 5] catalogue coverage task by task; we instead map metrics by operationalization and scale type, exposing where identical labels denote different quantities. TAMAS [56] and TRiSM [80] enumerate adversarial risks and governance requirements; we connect those categories to the federated-learning assumptions that agentic systems inherit, and show where local defenses fail to compose. Beyond Self-Talk [81] surveys communication-centric designs; we treat communication as a constraint layer whose latency, bandwidth, and ambiguity assumptions are usually left implicit, and we tie those assumptions to the evaluation and security layers. Across all five, the difference is not coverage but the cross-layer coordinate system through which the reviewed work is compared.

The present review addresses these three blind spots jointly: it organizes prior work by the perspective each stream adopts, proposes an integrative account of how collaboration fails and how errors propagate, and argues for metric alignment as a precondition for cumulative progress. In doing so it departs from existing surveys in scope, in the centrality it assigns to failure and adversarial behavior, and in its insistence that evaluation fragmentation is a structural obstacle rather than a incidental inconvenience.

- [1] Agent Drift: Quantifying Behavioral Degradation in Multi-Agent Systems — formalizes drift into semantic, coordination, and behavioral subtypes, providing the failure vocabulary this review builds on.
- [56] TAMAS: Benchmarking Adversarial Risks in Multi-Agent LLM Systems — a systematic attacker model for multi-agent LLM systems, spanning prompt, agent, and environment injection layers.
- [78] LLM-based Multi-Agent Systems: Techniques and Business Perspectives — proposes a coordination-form taxonomy and star/ring/graph/bus communication modes, a key comparator for architectural classification.
- [7] LLM Multi-Agent Systems: Challenges and Open Problems — enumerates layered, nested, and dynamic organizational structures and the open problems that motivate this review.
- [4] From LLM Reasoning to Autonomous AI Agents: A Comprehensive Review — assembles the benchmark inventory and the four-way capability grouping of MMLU, MATH/GSM8K, HumanEval/MBPP, and Spider/BIRD (with WikiSQL as an earlier precursor).
- [6] A Survey on Agent Workflow — Status and Future — catalogues agent workflow patterns and is the closest existing synthesis of orchestration control flow.
- [3] Exploring Autonomous Agents: A Closer Look at Why They Fail When Completing Tasks — documents empirical failure patterns that ground the robustness discussion.
- [52] Assessing and Enhancing the Robustness of LLM-based Multi-Agent Systems Through Chaos Engineering — adapts chaos engineering to stress-test multi-agent robustness.

- [27] AgentCoord: Visually Exploring Coordination Strategy for LLM-based Multi-Agent Collaboration — identifies natural-language ambiguity as a structural coordination hazard.

4 SURVEY METHODOLOGY

We assembled a corpus of 189 publications through multi-source retrieval, prioritizing arXiv and using open scholarly indexes as a fallback, with deduplication anchored on DOI or equivalent identifiers. This data mapping fixed both the provenance and the boundaries of the evidence base before any analysis began: five research questions were decomposed into 15 sub-questions, and the corresponding evidence set was frozen prior to analysis so that no later retrieval could reshape the material under examination. Of these 189 publications, 105 are cited in the synthesis; the remaining 84 informed screening and background and are not individually cited, so the reference list should be read as the cited subset of the screened corpus rather than as a census of it.

Refinement proceeded in two stages. First, inclusion was governed by the pre-specified sub-question structure, so that each retained publication had to bear on at least one of the 15 sub-questions; identifier-anchored deduplication removed duplicate records across sources. Second, evidence was fixed through a evidence-coding protocol that converted each publication into six evidence categories—problem, method, dataset or benchmark, metric, limitation, and assumption or constraint—together with cross-work links. This evidence-category-and-link representation served as the stable evidentiary record against which all subsequent claims were checked.

Evaluation operated on three layers. At the evidence layer, we constructed a evidence matrix over the six evidence categories and their cross-work links, yielding a structured view of how problems, methods, benchmarks, metrics, limitations, and assumptions connect across the 189 publications. At the analytical layer, each of the 15 sub-questions was addressed through comparative operations—classification, comparison, evolution tracing, and trade-off analysis—so that answers were derived from the fixed evidence set rather than asserted. At the verification layer, every conclusion was decomposed into individually checkable claims, and each claim was annotated with the specific publications that support it, allowing any statement to be traced back to its evidentiary basis.

This three-layer design separates what was collected (189 publications, deduplicated by identifier) from how it was structured (six evidence categories plus cross-work links) and from what was concluded (claims with explicit publication-level support). The separation matters because it makes the evidential status of each finding inspectable: a claim either has supporting publications in the fixed set or it does not, and the comparative operations that produced it are recorded alongside it.

5 PROPAGATION MECHANISMS OF ERRORS IN AGENT COLLABORATION

Having completed the evidence grading and traceability framework, the next task is to apply this framework to specific research topics and examine how it supports testable conclusions within actual literature collections.

Having established the methodological apparatus, we now turn to how errors originate and travel through multi-agent LLM collaboration. The chapter's central tension is that error propagation is simultaneously a named, recurring phenomenon—agent drift, constraint drift, and cascading hallucination form its recognizable backbone—and a process whose suppression mechanisms are validated under evaluation conditions so heterogeneous that their comparative effectiveness remains unsettled. We first characterize the trigger conditions and observable signatures of these named failure modes, then examine the mechanisms proposed to suppress propagation and anchor agent behavior, specifying the collaboration stage at which each intervenes. We close by assessing the strength and reproducibility of the quantitative evidence behind these mechanisms, a stratification that exposes how unevenly the field has established what actually works—and that sets up the architectural question of how task orchestration and role division trade off against one another.

5.1 Named Failure Modes in Multi-Agent Collaboration: Triggers, Manifestations, and Coupling

What named failure modes emerge during collaboration, and under which conditions do they arise and how do they manifest? The literature names 30 distinct modes, which we group into six families (Table 1): behavioral drift, constraint drift, cascades, coordination and communication failures, planning and termination failures, and governance failures. Three of these families—behavioral drift, constraint drift, and cascades—form the main trunk of behavioral degradation; the coordination/communication, planning/termination, and governance families describe the interaction-level conditions under which those

failures arise. The modes are discussed in that order below. The coordination/communication family is developed through role-induced conformity and inter-agent conflict; the planning/termination and governance families are developed at the end of the subsection. The boundaries between modes remain contested and their interactions are only partially characterized. Concretely, the twelve most consistently named headline labels (a subset of the inventory in Table 1) are:

1. agent drift (behavioral drift);
2. constraint drift (constraint drift);
3. error cascade (cascade);
4. cascading hallucination (cascade);
5. hallucination amplification (cascade);
6. role-induced conformity (coordination and communication);
7. inter-agent conflict (coordination and communication);
8. improper task decomposition (planning and termination);
9. failed self-refinement (planning and termination);
10. unrealistic planning (planning and termination);
11. maximum rounds exceeded (planning and termination);
12. opaque collaboration mechanisms (governance).

Table 1 places them within the six families; Table 1 enumerates the full inventory of 30 named modes by family, of which these twelve are the most frequently named headline labels; the twelve are not a disjoint partition, since three of them are cascade variants. The per-family counts are 4, 5, 3, 11, 4, and 3, summing to 30. MAST's 14-mode taxonomy [79] is one of the sources aggregated into this inventory; the twelve headline modes are those named most consistently across sources, not a separate taxonomy. We use "cascades" as the family name; error cascade, cascading hallucination, and hallucination amplification are its named modes. They share four interface-level vulnerabilities: intent loss at delegation (drift and its semantic and coordination variants), constraint erosion at memory and authority updates (constraint drift), state divergence at handoff (error cascade and hallucination amplification), and utility-induced weakening under optimization pressure (utility-induced drift).

Behavioral degradation over extended interaction is the most systematically quantified of these modes. Agent Drift formalizes the phenomenon as the gradual erosion of an agent's adherence to its assigned role and task constraints across successive turns, introducing drift metrics that separate semantic drift (deviation in content) from coordination drift (deviation in inter-agent protocol) [1]. The trigger is cumulative context accumulation rather than any single erroneous turn, and the manifestation is a monotone decline in task fidelity that becomes pronounced only after many exchanges—a signature that distinguishes drift from acute failure and implies that short-horizon evaluations systematically underestimate it. Agent Drift's simulation evidence quantifies this pattern: detectable drift (ASI below 0.85) appears at a median of 73 interaction rounds; within the first 100 rounds the ASI falls by 0.08 per 50 rounds, accelerating to 0.19 per 50 rounds between rounds 300 and 400; over the same horizon task success drops by 42%, human intervention rises 3.2-fold, and inter-agent conflict rises by 487.5%. Susceptibility is domain-dependent: financial analysis is most affected (53.2% within 500 rounds), followed by compliance monitoring (39.7%) and enterprise automation (31.8%) [1]. These figures come from simulation rather than production deployment, a limitation the same line of work acknowledges; they nonetheless establish that drift is gradual, accelerating, and measurable, which is why short-horizon evaluations understate it.

A closely related but mechanistically distinct mode is constraint drift, which concerns the erosion of safety and behavioral constraints rather than role fidelity. The central claim in this line of work is that safe multi-agent behavior must be continuously maintained rather than asserted once at initialization; constraints that hold at deployment can decay as agents negotiate subtasks and reinterpret instructions [2]. This contrasts instructively with behavioral degradation: drift of the first kind is driven by context length and is largely task-agnostic, whereas constraint drift is driven by the semantic pressure of inter-agent exchange and surfaces specifically where agents must reconcile conflicting subgoals. The two may

co-occur, but their differing triggers suggest that mitigations targeting one will not automatically suppress the other.

Role specialization introduces a third mode in which the structure intended to improve reliability itself becomes a failure source. In role-specialized pipelines, agents assigned narrow responsibilities can propagate errors with high confidence because downstream roles lack the authority or context to challenge upstream outputs; traceability analyses show that accountability becomes diffuse precisely at role boundaries, making the origin of a bad decision hard to localize after the fact [8]. MetaGPT's standardized operating procedures partially mitigate this by enforcing structured intermediate artifacts that make handoffs inspectable [14], which indicates that the failure is a property of unstructured handoffs rather than of specialization per se.

Conformity and premature consensus constitute a fourth mode, observable when agents converge on a shared but incorrect answer. Studies of embodied agents in organized teams report that explicit role assignment and organizational structure improve cooperative outcomes, yet also create conditions under which lower-status agents defer to a designated leader even when the leader's plan is flawed. Theory-of-mind evaluations sharpen this picture: agents exhibit only partial models of collaborators' beliefs, and errors in inferred intent lead to coordination failures that resemble conflict but originate in misrepresentation rather than disagreement. The manifestation is therefore deceptive—surface agreement coexists with latent divergence, and the failure only becomes visible when the shared plan is executed.

Inter-agent conflict proper—explicit disagreement or deadlock—appears to be the least frequent but most visible mode, and its triggers differ from those of conformity. Temporal-graph monitoring of multi-agent conversations detects anomalous interaction patterns indicative of escalating disagreement and coordination breakdown before task failure becomes terminal [82]. Communication delay and misaligned timing represent an orthogonal trigger: when intent and timeliness are not jointly aligned, agents act on stale representations of one another's state, producing conflict that is procedural rather than substantive. This suggests that a substantial share of apparent disagreement is an artifact of synchronization rather than genuine divergence of goals.

Beyond these named modes, failure-taxonomy work on autonomous agents supplies the underlying error classes from which collaborative failures are composed. Analyses of task-completion failures identify planning errors, tool-use errors, and self-verification failures as dominant categories, and in multi-agent settings these individual-level errors are amplified through handoffs rather than averaged away. Hierarchical frameworks that integrate memory into task allocation attempt to intercept this amplification by giving allocating agents access to prior failure context, and domain deployments such as clinical decision support show that the same error classes recur with higher stakes. The consistency of these categories across domains indicates that the named collaborative modes are specializations of more general agent failure classes.

Planning and termination failures form the fifth family. Analyses of task-completion failures identify improper task decomposition, failed self-refinement, unrealistic planning, and maximum-rounds-exceeded as recurrent categories; in multi-agent settings these errors are amplified through handoffs rather than averaged away, because a decomposition error made upstream becomes an infeasible subtask downstream [3]. The sixth family is governance-level: opaque collaboration mechanisms, missing incentive structures, and limited scalability explain why some multi-agent systems fail even when each individual agent behaves competently, since coordination is unobservable, incentives favor self-reported success, and fixed protocols do not scale [83, 80]. These two families matter for the stack because they are the modes least visible to per-agent evaluation and most likely to be misattributed to model capability.

The coupling among modes is the least resolved aspect of this literature. Drift and constraint drift can compound, since prolonged interaction both lengthens context and multiplies opportunities for constraint reinterpretation [1, 2]; conformity can mask drift by suppressing the dissent that would otherwise reveal it. Detection efforts accordingly shift from static assertions to temporal monitoring, on the premise that coupled failures are visible only in interaction traces [82]. Incentive and transparency mechanisms address a complementary trigger—strategic misrepresentation—by making agent contributions auditable, while architecture-search systems demonstrate that structured multi-agent deliberation can be productive when roles and evaluation criteria are tightly specified. Chaos-engineering approaches invert the framing by deliberately injecting faults to surface latent failure modes before deployment [84].

Taken together, the evidence supports a taxonomy in which behavioral degradation and constraint drift are gradual, context-driven erosions; role-induced conformity and misrepresented intent are structural, specialization-driven distortions; and explicit conflict is an acute, often synchronization-driven event. The modes differ in trigger, timescale, and observability, which explains why no single detection strategy covers them all. This taxonomy provides the vocabulary needed to ask how these failures propagate once initiated—the question taken up in the following section.

Table 1: Named failure modes grouped into six families. The table makes the count of named modes reconstructable: each family lists its constituent modes, and the right column gives representative evidence.

Family	Named modes	Count	Evidence
Behavioral drift	agent drift; semantic, coordination, and behavioral drift	4	[1]
Constraint drift	memory, authority, information-flow, accountability, and utility-induced drift	5	[2]
Cascade	error cascade; cascading hallucination; hallucination amplification	3	[8]
Coordination and communication	duplicated message; repeated command; conflicting command; conflicting plans; improper delegation; ignoring requests; behavioral conflicts; memory inconsistency; asynchronous communication; role-induced conformity; inter-agent conflict	11	[85, 9, 62]
Planning and termination	improper task decomposition; failed self-refinement; unrealistic planning; maximum rounds exceeded	4	[3]
Governance	opaque collaboration mechanisms; missing incentive structures; limited scalability	3	[83]

5.2 Stage-Targeted Suppression and Behavioral Anchoring in Collaborative Workflows

If the preceding taxonomy explains how errors originate and propagate, a natural question follows: what mechanisms have existing systems introduced to suppress propagation or anchor agent behavior, and at which stage of the collaborative workflow do they intervene? Across the literature, interventions cluster into three stage-specific groups—pre-execution structural constraints, in-execution behavioral monitoring, and post-hoc traceability—and the evidence suggests that each group trades coverage against cost in ways that mirror the propagation modes it targets.

The earliest point of intervention is the structural contract imposed before any agent acts. MetaGPT encodes standardized operating procedures and role-specific output schemas so that downstream agents receive structured artifacts rather than free-form chatter, which limits the surface through which one agent's error can corrupt another's input [14]. This is an anchoring mechanism in the strict sense: behavior is constrained by the interface, not by runtime supervision. A comparable logic appears in hierarchical task allocation, where memory-integrated decomposition assigns subtasks and their acceptance criteria up front, reducing the ambiguity that lets a faulty intermediate result be silently absorbed [9]. Both approaches act at the planning stage and share a common limitation—they anchor the form of inter-agent communication but cannot detect a well-formed yet incorrect payload, which is precisely the failure mode that later-stage monitors are built to catch.

In-execution monitoring constitutes the second family and intervenes while the workflow is running. GUARDIAN applies temporal graph learning over the interaction trace to flag anomalous collaboration patterns as they emerge, treating the sequence of agent exchanges rather than any single message as the unit of analysis [82]. This temporal framing is significant because it targets the synchronization-driven, acute failure modes that static schema checks cannot see. However, the same design choice imposes a detection latency: the anomaly must manifest across several interactions before the graph signal strengthens, so GUARDIAN is better suited to containing an already-spreading error than to preventing its first emission. The contrast with MetaGPT is instructive—schema anchoring is cheap and immediate but blind to semantic corruption, whereas temporal monitoring is semantically sensitive but reactive.

A third family anchors behavior through organizational and incentive structure rather than technical enforcement. Embodied agents organized into explicit teams with designated leaders show improved cooperative outcomes, indicating that role hierarchy itself functions as a behavioral anchor by concentrating decision authority [85]. Theory-of-mind capabilities similarly allow agents to model collaborators' beliefs and thereby reduce miscoordinated actions that seed propagation. These mechanisms operate at the coordination stage and are notably softer than schema or graph constraints: they shape the probability of erroneous behavior rather than blocking it. This distinction matters for propagation analysis, because soft anchoring degrades gradually under extended interaction, a phenomenon documented as behavioral drift over long horizons [1].

The fragility of soft anchoring is sharpened by evidence that safety properties in multi-agent systems are not preserved by assertion alone. Constraint drift has been observed in LLM-based multi-agent systems,

where declared safety constraints gradually lose their hold on agent behavior as interactions accumulate, implying that anchoring mechanisms must be actively maintained rather than specified once [2]. This finding cuts across the families above: a schema, a monitor, and a role hierarchy all face the same erosion problem, but only the monitor is positioned to observe the erosion as it happens. The implication is that pre-execution anchoring and in-execution monitoring are complements rather than substitutes, and systems relying on either alone leave a predictable gap.

Post-hoc traceability forms the final family, intervening after execution to localize where a fault entered the pipeline. Role-specialized pipelines instrumented for accountability can attribute outputs to specific agents and stages, which supports root-cause analysis once propagation has already occurred. Blockchain-driven transparency mechanisms extend this logic to decentralized settings, using immutable records to make collaboration auditable and incentives compatible [83]. These mechanisms do not suppress propagation in real time; their value is diagnostic and deterrent. A useful contrast emerges here: traceability is the only family that scales to decentralized topologies where no central monitor observes all exchanges, yet it offers no protection during the window in which an error is actively spreading.

Communication-level design offers a further anchoring strategy aimed at the interface itself. CoDe addresses communication delay by aligning intent and timeliness between sender and receiver, so that stale or misaligned messages are reconciled rather than blindly consumed. This targets a specific propagation channel—temporal misalignment—that neither schema constraints nor role hierarchies address directly. The mechanism is best understood as filtering at the boundary: it reduces the probability that a delayed message is misinterpreted as current, which is a common trigger for the synchronization-driven failures identified earlier. Its scope, however, is confined to timing and intent alignment; semantic errors that arrive on time and with clear intent pass through unaffected.

Domain deployments illustrate how these families combine under real constraints. In automated chaos engineering, LLM-driven agents must coordinate fault injection while preserving system resilience, and the workflow embeds verification steps between planning and execution to catch erroneous actions before they affect production. In clinical decision support, ColaCare routes specialist agents through a structured consultation protocol with an aggregating moderator, which anchors individual agent outputs against a synthesis stage. Both cases place a verification checkpoint at a stage boundary rather than monitoring continuously, suggesting that stage-gated verification is a pragmatic compromise when full temporal monitoring is too costly. The trade-off is coverage: errors that propagate entirely within a single stage escape the checkpoint.

Neural architecture design offers a contrasting configuration in which the collaborative workflow itself is the object of verification. NADER coordinates multiple agents to propose and evaluate architecture candidates, and the evaluation stage functions as an intrinsic anchor because proposals must survive empirical assessment before being accepted. Here the anchoring mechanism is the task's own feedback signal rather than an external monitor, which is efficient when ground-truth evaluation is cheap but unavailable in open-ended tasks where no such oracle exists. This boundary condition is important: the effectiveness of verification-based anchoring is contingent on the availability of an evaluable objective, and the literature offers little evidence on how it performs when that condition fails.

Failure analyses reinforce the stage-specificity of these mechanisms. Studies of autonomous agent failures find that a substantial share of breakdowns arise from planning and coordination errors rather than execution faults, which implies that pre-execution anchoring addresses the largest single error source [3]. Yet the same analyses show that failures compound across stages, so an error surviving the planning anchor enters execution where only runtime monitoring can intercept it. Read together with the drift and constraint-drift findings, this suggests a layered architecture in which schema constraints, temporal monitors, and traceability each cover a distinct segment of the propagation path [1, 2].

Synthesizing across the three trunk families, a consistent pattern emerges: mechanisms acting earlier in the workflow are cheaper and more general but semantically blind, while mechanisms acting later are semantically rich but reactive and costly. No single mechanism in the reviewed literature covers both the emission and the propagation of errors, and the empirical record—drift under long horizons, constraint erosion, compounding stage failures—indicates that anchoring must be maintained continuously rather than established once. This complementarity, rather than any individual mechanism's superiority, is the central finding of the present analysis. It also exposes a gap: few systems report quantitative suppression rates under controlled error injection, leaving the relative efficacy of these families an open empirical question that the following chapter takes up through comparative evaluation.

5.3 Quantitative Validation Conditions and the Strength of Efficacy Evidence

What evaluation conditions have been used to validate these suppression mechanisms, and how strong is the evidence for their effectiveness? The short answer is that quantitative comparison is universal but unevenly rigorous, and that the field's evaluation designs are so heterogeneous that cross-paper efficacy figures cannot be legitimately compared. Every suppression mechanism surveyed reports at least one controlled baseline, and none is a purely qualitative demonstration; yet the depth of that quantitative support varies sharply, and the conditions under which it was obtained differ along nearly every axis that matters for generalization.

The strongest evidence comes from studies that combine repeated runs, variance reporting, and large-scale sampling. NADER evaluates its architecture-design collaboration across 500 candidate architectures with three random seeds, reporting mean and standard deviation, and finds that 30.8% of generated designs surpass the best architecture in the NAS search space while 58.8% of proposals are executable [42]. ColaCare similarly reports AUROC and AUPRC with bootstrap-derived confidence intervals across three clinical datasets, and its ablation is unusually informative: removing the specialist models drops AUPRC from 54.05 to 27.73, isolating the contribution of the multi-agent decomposition rather than merely reporting an aggregate gain [86]. These designs approximate the reproducibility standard that controlled error-injection studies would demand, although neither actually injects errors; they measure end-task performance under normal operation.

A second tier of evidence rests on smaller samples and legacy automatic metrics. DesignGPT conducts a user study with 30 participants and two expert raters, reporting an inter-rater reliability of 0.90 and statistically significant advantages in completeness and feasibility, but the novelty and feasibility gains of 11.06%, 24.26%, and 39.90% are derived from a small, single-population sample [19]. MultiAgentESC likewise relies on 100 dialogues from ESConv, supplementing n-gram metrics with a five-dimensional human rating and a GPT-4o judge; the n-gram overlap measures are acknowledged in that line of work as unable to capture the true quality of emotional support [87]. The contrast with the first tier is instructive: both studies do include baselines and significance claims, but the sample sizes and metric provenance make their effect estimates considerably more fragile.

The conditions of validation diverge even more than the rigor does. The six mechanisms span neural architecture search (NADER), compositional text-to-image generation (MCCD), conceptual design collaboration (DesignGPT), clinical EHR prediction (ColaCare), long-document question answering (LONGAGENT), and emotional-support dialogue (MultiAgentESC). The six mechanisms are the evaluated systems; the three stage-specific groups describe where they intervene; and the five open problems discussed in the Future Research chapter are a research agenda that cuts across both, so the mapping is many-to-many rather than one-to-one. Pairwise Jaccard similarity between their metric sets is near zero, and the only non-trivial overlap arising where two studies share an F1 measure. This means the community has not converged on what counts as having suppressed a given error class; each mechanism is validated on its own task-specific or domain-specific benchmark. A direct comparison of a 9.04% overall gain in compositional image quality against an 18.9% accuracy improvement in long-document QA would therefore be illegitimate, since the two numbers describe different constructs measured on different instruments [43, 88].

Where mechanisms are evaluated within a shared paradigm, the reported effects are substantial but narrowly scoped. LONGAGENT reports an average 18.9% accuracy improvement from cross-member communication and a 16.42% advantage over GPT-4 across ten runs, and its qualitative observation that hallucinating members tend to correct their responses upon receiving a chunk containing the correct answer is a plausible mechanistic account of why communication helps [88]. MCCD reports a 9.04% overall improvement on T2I-CompBench with a 41.49% gain on spatial relationships, a pattern consistent with the idea that structured inter-agent critique primarily repairs compositional binding errors [43]. Both are convincing on their own terms, yet neither isolates suppression from general capability improvement, and neither reports the cost of the added coordination.

That cost is the most conspicuous omission. Only the accountability-oriented line of work quantifies the price of suppression alongside its benefit, reporting a two-to-threefold increase in cost and a rise in median latency from roughly two seconds to twenty to twenty-five seconds, an eight-to-tenfold slowdown [8]. The remaining studies report efficacy gains without corresponding overhead figures, which biases the apparent evidence in favor of suppression: a mechanism that improves accuracy by eighteen points while tripling inference cost is not obviously preferable to a simpler alternative. This asymmetry is a structural feature of the evidence base rather than an incidental reporting habit, and it limits any efficiency-aware comparison across mechanisms.

Coverage of evaluation dimensions is similarly skewed. Accuracy and success-rate metrics appear in

roughly two-thirds of the studies, whereas robustness to worst-case or out-of-distribution inputs, cross-subgroup fairness, and industrial-scale validation are absent throughout. The clinical work illustrates the risk: AUROC is known to overestimate performance on imbalanced data and must be supplemented by AUPRC and sensitivity-oriented measures, a caveat the authors themselves raise [86]. Success-rate metrics carry an analogous defect, since they collapse open-ended tasks into discrete answers and treat partial correctness as failure; this limitation has been explicitly acknowledged in agent-failure analyses and in traceability studies that note the absence of interaction-level and root-cause diagnostics [3, 8]. High-frequency metrics, in short, are not thereby valid metrics.

Theoretical guarantees and stress testing are entirely missing from the validation repertoire. All six mechanisms are validated through controlled experiments, but only some report ablations—ColaCare's removal of specialist models is the clearest—and none offers a formal argument that suppression holds under adversarial or distributional shift, and none employs simulation to probe failure boundaries. This is a meaningful gap for a chapter concerned with propagation: a mechanism that reduces average error may leave the tail of cascading failures untouched, and without worst-case evaluation there is no evidence either way. The absence is partly a maturity issue, since formal treatment of multi-agent error dynamics remains rare, but it also reflects a community preference for benchmark gains over boundary characterization.

It is important to state the limits of these conclusions honestly. The robustness, fairness, and industrial-scale gaps each rest on only one or two supporting studies, which falls short of the threshold for claiming systematic neglect; they are best described as suspected blind spots requiring broader sampling. The comparability estimate is computed at the level of metric sets and does not penetrate to differences in metric definitions, so it may understate heterogeneity. And the strength stratification itself is a qualitative judgment, since several papers do not fully report confidence intervals or multiple-comparison corrections.

Taken together, the evidence supports a qualified conclusion. Suppression mechanisms have been validated quantitatively, but only two of the six mechanisms examined here—NADER and ColaCare—meet a strong standard of repeated runs, variance reporting, and evaluation at scale; the remaining four (DesignGPT, MultiAgentESC, LONGAGENT, and MCCD) provide weaker or narrower evidence. The six are discussed in tier order; LONGAGENT and MCCD are treated as single-benchmark lower-tier evidence, which is why their validation conditions are summarized in Table 2 rather than expanded at the same length. DesignGPT rests on a 30-participant user study with inter-rater reliability 0.90; MultiAgentESC on 100 dialogues with a five-dimensional human rating; LONGAGENT on ten runs over one benchmark; and MCCD on a single benchmark without repeated-run variance. None of the four reports multi-dataset evaluation.

Table 2: Evidence standards for the six suppression mechanisms. Only NADER and ColaCare meet all three criteria of repeated runs, variance reporting, and evaluation at scale; NADER's scale criterion is 500 candidate architectures within one search space, and ColaCare's is three clinical datasets. The remaining four meet the criteria partially or not at all.

Mechanism	Domain	Repetition	Variance	Scale	Tier
NADER	Neural architecture search	3 seeds	Mean and std.	500 architectures, one space	Strong
ColaCare	Clinical EHR prediction	Bootstrap	Confidence intervals	3 datasets	Strong
DesignGPT	Conceptual design	User study (30)	Inter-rater 0.90	Single population	Moderate
MultiAgentESC	Emotional support	100 dialogues	5-dim. rating	Single dataset	Moderate
LONGAGENT	Long-document QA	10 runs	Not reported	Single benchmark	Weak
MCCD	Text-to-image	Not reported	Not reported	Single benchmark	Weak

Validation has occurred under conditions that are task-idiosyncratic, cost-blind, and confined to average-case performance. The field can say that these mechanisms work on their own benchmarks, and cannot yet say how well they work relative to one another, at what cost, or under what adversarial conditions. Establishing that comparative picture—through shared benchmarks, controlled error injection, and joint reporting of benefit and overhead—is the task to which the following chapter turns.

The taxonomy therefore constrains the architecture layer: a design that cannot detect or contain these modes is inadmissible, and the architecture chapter evaluates which designs remain admissible.

6 TASK ORCHESTRATION TRADE-OFFS IN COLLABORATION ARCHITECTURES AND ROLE DIVISION

Since comparable evidence for the efficacy of the various mechanisms is still lacking, measuring their relative merits, cost levels, and robustness under adversarial conditions requires a common evaluation benchmark and error-injection methodology; the following turns to the construction of this comparative framework.

Building on the preceding analysis of how errors propagate through agent collaboration, this chapter turns from diagnosis to design and examines the orchestration trade-offs that shape collaboration architectures and role division. The central tension is that no single architecture dominates: centralized topologies with fixed roles systematically favor task accuracy, traceability, and error attributability, whereas decentralized, self-organizing alternatives favor scalability and cross-organizational adaptability, with both bound by an inescapable accuracy–cost/latency–scalability trilemma. The chapter first reconstructs the fragmented design space into a coherent set of design dimensions, then characterizes the conditional, Pareto-frontier nature of the centralization–decentralization and fixed–evolving role choices, and shows why hybrid designs often occupy the efficient frontier. It closes by tracing how these architectural choices shape what can be measured, motivating the fragmentation of evaluation benchmarks and the call for cross-task unification taken up in the next chapter.

6.1 Design Dimensions of Multi-Agent Collaboration Architectures

How can existing collaboration architectures be classified along principled design dimensions? Prior classifications propose three axes—orchestration topology, role allocation, and explicit standard operating procedure (SOP) structuring—but we retain only two principal dimensions; SOP structuring is a within-cell implementation choice. We therefore use the 2×2 space of topology and role regime in Table 3, and read SOP use as a property of each cell: centralized topologies tend to encode SOPs explicitly, while decentralized ones rely on emergent decomposition. This reduction is what makes the design space comparable; a third independent axis would multiply categories without adding discriminative power, because SOP use is largely determined by the topology–role combination.

Orchestration topology forms the most extensively documented axis, spanning a spectrum from centralized to fully decentralized control. Centralized designs appoint a coordinating agent or planner that decomposes tasks and dispatches subtasks, whereas decentralized designs allow agents to discover and negotiate peers directly. AgentNet exemplifies the decentralized extreme, organizing agents in a dynamically evolving network where coordination emerges from local interactions rather than a global controller [89]. A survey of LLM-based multi-agent systems similarly distinguishes hierarchical, cooperative, and competitive organizational forms, noting that the choice of topology constrains which coordination protocols remain feasible [78]. The central–decentralized distinction therefore functions less as a binary label than as a design parameter with downstream consequences for communication cost and failure containment.

A second, finer-grained topological dimension concerns how the agent set itself evolves during execution. Static architectures fix the roster of participating agents at design time, while dynamic architectures add or remove agents in response to task demands. AgentDropout operationalizes this by eliminating redundant agents during collaboration, reporting simultaneous reductions in token consumption and improvements in task performance relative to fixed-roster baselines [90]. Evolving orchestration pursues a related goal at the level of the coordinator, adaptively reshaping the orchestration policy as collaboration proceeds [91]. In contrast, cross-team orchestration extends the topology horizontally, coordinating multiple teams rather than individual agents and thereby introducing an additional tier of hierarchical structure. These three approaches share a commitment to adaptivity but differ in what is allowed to change: the agent set, the orchestration policy, or the team composition.

Role allocation constitutes the second major axis and admits its own subdivision into predefined versus emergent assignment. Predefined allocation assigns each agent a fixed persona or functional specialization at initialization, a pattern common in business-oriented deployments where responsibilities map onto recognizable organizational functions [78]. Emergent allocation instead lets roles crystallize through interaction, as in decentralized evolutionary coordination where agents specialize without an external assignment step. The trade-off is legible: predefined roles yield predictable interfaces and easier debugging, whereas emergent roles may discover specializations that designers did not anticipate, at the cost of convergence uncertainty. Reward attribution decomposition offers a complementary lens, showing that how credit is assigned across agents implicitly encodes assumptions about role contribution and thus shapes the roles that evolve.

The third axis, SOP structuring, distinguishes architectures that embed explicit procedural knowledge

from those that rely on emergent task decomposition. Flow-of-Action demonstrates the structured end of this spectrum, encoding root-cause-analysis workflows as SOPs that constrain agent behavior and thereby reduce deviation from expert practice [92]. Workflow-oriented surveys corroborate that explicit process specifications improve reproducibility and auditability in agent pipelines [6]. However, the same rigidity that suppresses deviation may also suppress adaptation when task conditions depart from the SOP's assumptions, a tension that unstructured emergent decomposition avoids at the price of variance. The degree of SOP structuring therefore trades predictability against flexibility, and the appropriate setting appears contingent on how well the task domain can be specified in advance. In the framework used here, SOP structuring is therefore a within-cell design choice rather than a third dimension.

Cross-cutting the topology and role dimensions is the communication substrate, which prior multi-agent reinforcement learning research treats as a first-class design variable rather than an implementation detail. A survey of multi-agent deep reinforcement learning with communication catalogues how communication topology, bandwidth, and learned messaging protocols interact with coordination performance [20]. This perspective anticipates a recurring finding in LLM-based systems: token cost scales with the density of inter-agent communication, so topological choices have direct economic consequences. Communication design thus couples the topology and role axes, since denser connectivity and more numerous roles both inflate message volume.

Quantitative evidence sharpens the comparison between centralized and decentralized orchestration. Scaling studies of LLM-based collaboration indicate that performance gains from adding agents are sublinear and can reverse beyond a threshold, suggesting that centralized coordination with a bounded agent pool may outperform unbounded decentralization on tasks with tight interdependencies. Decentralized evolutionary coordination, by contrast, reports advantages in adaptability and reduced reliance on a single coordinator, though at the cost of less predictable convergence [89]. Neither result generalizes unconditionally; the divergence is best read as evidence that topology suitability is task-contingent rather than universally ordered.

Role allocation exhibits a parallel contingency. Fixed-role architectures simplify credit assignment and error localization, which matters in domains where accountability is required, such as federated analytics over decentralized data sources where each agent's contribution must be traceable. Emergent-role architectures, conversely, have been observed to discover efficient specializations in open-ended problem solving, but the absence of predefined interfaces complicates both debugging and reward decomposition. A reasonable synthesis is that predefined roles dominate when the task decomposition is known and stable, while emergent roles dominate when it is not—a conditional that the current literature states qualitatively but rarely quantifies.

SOP structuring shows a similar conditional pattern with an additional failure mode. Structured SOPs reduce variance and improve alignment with expert workflows in diagnostic tasks [92], and workflow surveys report that explicit specifications ease integration with existing enterprise processes [6]. Yet over-specification can produce brittle behavior when inputs violate SOP preconditions, and the literature documents few mechanisms for gracefully relaxing procedural constraints mid-execution. This gap is notable because it mirrors the rigidity–adaptability trade-off observed at the topological level, hinting that the two dimensions may share an underlying cause: the cost of committing to structure before task uncertainty is resolved.

Several open problems cut across all three design considerations. A survey of challenges in LLM multi-agent systems identifies coordination overhead, error propagation across agent boundaries, and the difficulty of evaluating collective rather than individual performance as persistent obstacles [7]. These challenges are dimension-sensitive: error propagation is amplified by dense topologies, misallocated roles complicate attribution, and rigid SOPs can mask failures until late in execution. The literature nonetheless lacks a unified framework that predicts, for a given task profile, which combination of topology, role allocation, and SOP structuring will minimize these costs—an absence that motivates the trade-off analyses taken up in the following section. Table 3 summarizes the resulting design space.

6.2 The Centralized–Decentralized Trade-off in Task Orchestration

Does the evidence support a single superior orchestration route, or does the choice between centralized control (SOPs, fixed roles) and decentralized coordination (evolving roles) depend systematically on task type, scale, and robustness requirements? The evidence assembled here points to the latter: the two routes form a conditional Pareto structure in which no architecture dominates across all three performance dimensions, and the most defensible conclusion is that each route buys performance on one axis by conceding another [92, 89].

Table 3: The two-dimensional design space of multi-agent collaboration. Representative systems are placed by orchestration topology and role regime; the axes interact through communication cost rather than varying independently.

Role regime	Centralized topology	Decentralized topology
Predefined	SOP-driven pipelines: MetaGPT [14], Flow-of-Action [92], orchestrator–worker hierarchies [78]. Strength: accuracy and auditability.	Static peer topologies (ring, bus, DAG) [93]. Strength: scalability; weakness: error attribution.
Evolving	Adaptive coordinator with a dynamic roster [91, 90]. Strength: task adaptation; weakness: convergence uncertainty.	AgentNet-style evolving networks with emergent specialization [89]. Strength: adaptability; weakness: predictability.

For tasks with an established diagnostic or compliance procedure, centralized SOP orchestration yields large and reproducible accuracy gains. Flow-of-Action encodes the SRE standard operating procedure as an explicit SOP flow and raises root-cause-analysis accuracy from 35.50% under ReAct to 64.01% with GPT-4-Turbo; its ablations are more telling still, since removing SOP knowledge collapses average accuracy from 54.06% to 15.39% and removing the SOP flow drops localization accuracy from 54.22% to 15.11% [92]. The mechanism is constraint rather than capability: the fixed procedure narrows the trajectory space in which the agents can hallucinate. Comparable patterns appear in legal reasoning, where a role-aligned mixture-of-experts architecture mirroring law-firm workflow outperforms GPT-4 on LawBench by 7.73 percentage points, and in code generation, where SOP-encoded assembly lines reach 85.9%/87.7% Pass@1 on HumanEval/MBPP [14]. Centralization, in short, is most valuable exactly where the task itself supplies the structure.

However, the same rigidity that produces these gains becomes a liability when the task lacks a predefined procedure. AgentNet argues explicitly that static, predefined workflows cannot absorb real-time fluctuations in agent performance or rapidly shifting task requirements, and the authors of SOP-based assembly lines concede that such pipelines struggle with the diversity of real-world applications and fail to cover UI and front-end scenarios [89]. This is a boundary condition rather than a refutation: SOPs appear to function as a strong prior that must be matched by the task, and their advantage attenuates as task openness increases. The contrast with evolving-orchestration approaches is therefore not about which is more capable in general, but about which prior—procedure or adaptation—the task distribution rewards.

Scalability evidence sharpens this contrast and, notably, undercuts both pure routes. A four-framework comparison spanning centralized (CMAS), decentralized (DMAS), and two hybrid designs across 4–32 robots and four coordination-dependent tasks found that the hybrid HMAS-2, combining central planning with local feedback, achieved the highest success rate on all four tasks with the fewest steps at 3–6 robots, whereas DMAS consumed the most API calls and tokens while attaining the lowest success rate, with agents repeatedly restating one another and diluting critical context. Pure centralization was competitive only at small scale and degraded markedly as agent count grew, trailing HMAS-2 on the complex warehouse task at every scale. The scaling law reported for LLM collaboration reinforces the ceiling: performance saturates sigmoidally with agent count, with roughly $2\hat{}4$ nodes a reasonable operating point, and centralized topologies exhibit frequent context explosion beyond about thirty agents [93].

The decentralized advantage is itself conditional on team size, which complicates the intuition that heterogeneity always helps. AgentNet's own experiments show that with three agents a fully homogeneous team is optimal (0.86), and introducing diversity lowers performance to 0.81; heterogeneity becomes advantageous only at five agents (0.85 versus 0.79) [89]. A plausible interpretation is that coordination overhead from heterogeneous roles is a fixed cost that must be amortized over enough parallel work, so small teams pay the cost without reaping the benefit. This size-dependence mirrors the centralization result from the opposite direction: centralized structures are efficient when the team is small and the task is structured, while decentralized structures become efficient only once scale generates enough diversity to offset coordination cost.

Robustness evidence is the weakest of the three performance dimensions and must be read with corresponding caution. Structured handoff has been shown to arrest failure cascades, lifting accuracy on PythonIO from 61.42% to 97.64%, a gain of 36.22 points, yet the same study observed that strict handoff can occasionally restrict downstream agents' flexibility and cause small accuracy regressions [8]. The costs of centralization are quantifiable and substantial: accountable pipelines raise cost by roughly 2–3× and median latency from about 2 s to 20–25 s, an 8–10× slowdown, and the central orchestrator constitutes

a single point of failure [8]. Decentralized designs trade coordination determinism for fault tolerance and privacy, as AgentNet's DAG routing and retrieval-based memory are intended to do, while dynamic agent elimination has been reported to improve performance and token efficiency across layered and random initial topologies [89, 90].

These observations converge on a three-way constraint: no configuration simultaneously maximizes task accuracy, minimizes cost and latency, and scales linearly without bound in agent count. Centralized SOPs buy accuracy at the price of cost, latency, and brittleness to novel inputs; decentralized coordination buys scalability and fault tolerance at the price of coordination overhead and accuracy variance in small teams [91, 89]. The literature does not resolve this constraint so much as map its frontier, and the mapping is uneven: accuracy and cost are comparatively well instrumented, whereas robustness remains an emerging dimension supported by too few independent measurements to enter a formal comparison [7].

A further complication is that the route debate is not merely empirical but architectural, and the two camps conflict directly. Reinforcement-learned orchestration has been reported to improve both performance and token efficiency, but it requires a tunable weight to trade accuracy against cost and remains a centralized design—placing it in direct tension with decentralized proposals [91, 89]. Stronger LLM orchestrators likewise raise accuracy while raising cost in step, so orchestration quality and orchestration expense are coupled rather than independent levers [78]. This coupling suggests that some of the apparent disagreement between camps may reflect task differences rather than architectural essence, a possibility the current evidence cannot adjudicate.

Cross-paper comparison is further constrained by benchmark heterogeneity. The relevant studies evaluate on GoogleOnlineBoutique, PythonIO, HumanEval, MBPP, LawBench, and BoxNet using incompatible metric definitions—localization accuracy, Pass@1, and task success rate are not interchangeable—so numerical results must be interpreted within each paper's own context rather than pooled into a single ranking [92, 93]. Methodological dependencies compound the problem: SOP-based diagnosis relies on matching historical incidents, and hybrid advantages have been validated mainly on warehouse-style tasks whose generality remains untested [92, 93]. These limitations do not overturn the directional findings, but they counsel against treating any single reported margin as a general law of architecture.

Taken together, the evidence supports a conditional rather than a universal recommendation. Structured, high-compliance tasks such as root-cause analysis, legal reasoning, and code generation favor centralized SOP flows, accepting construction cost and dependence on historical matching; large-scale, cross-organizational, or privacy-sensitive settings favor decentralized coordination, accepting small-team coordination overhead and accuracy fluctuation; and cost-sensitive settings that still demand high accuracy are best served by heterogeneous accountable pipelines that sit on the accuracy–cost frontier [92, 89, 7]. Hybrid designs, which combine central planning with local feedback, appear on the Pareto frontier in most controlled comparisons, suggesting that the practical resolution is not to choose a camp but to choose a mixing ratio [93].

The clearest gap follows directly from this synthesis: centralized and decentralized routes have not been compared under a shared protocol on robustness, and the one review-oriented effort in this space has not yet produced quantitative results [7]. Until such head-to-head evidence exists, claims that either route is intrinsically more robust remain provisional. This unresolved robustness question, together with the cost and latency penalties that accompany every accuracy gain documented above, sets up the next question: how role division and communication structure can be designed to shift the frontier itself rather than merely move along it.

6.3 Role Regimes and Their Consequences for Coordination Cost and Adaptivity

How does the choice between predefined, fixed roles and dynamically evolving roles shape the performance of multi-agent collaboration, and what design mechanisms underlie each regime? The literature converges on a conditional answer: fixed role assignment buys coordination economy and interpretability, whereas role evolution buys adaptivity and robustness, and the boundary between the two is set by task heterogeneity, team scale, and the cost of renegotiating structure mid-task.

Static role regimes derive their advantage from eliminating the allocation problem at run time. When roles are fixed, the orchestration burden collapses to routing and scheduling, and the system inherits the predictability of a hand-designed workflow; surveys of agent workflows accordingly treat role specification as a first-class design artifact rather than an emergent property [6]. This economy is most visible in domains where the task decomposition is itself standardized: SOP-driven root-cause analysis encodes diagnostic responsibilities directly into role definitions, so that each agent's scope is determined before any instance arrives [92]. The same logic appears in federated analytics, where role boundaries map onto data

boundaries and thereby constrain what each participant may legitimately observe [51]. In these settings the fixed regime is not a simplification but a correctness requirement, since role stability is what makes the division of labor auditable.

The cost of that stability is brittleness under distribution shift. A predefined role set encodes an implicit assumption about which subtasks will arise; when that assumption fails, the team has no mechanism for generating the missing competence, and the failure surfaces as either deadlock or silent degradation. General surveys of LLM-based multi-agent systems identify this mismatch between static organizational structure and open-ended task demands as a recurring open problem [7]. The difficulty is compounded by communication: multi-agent reinforcement learning work shows that fixed communication topologies, which typically accompany fixed roles, limit the propagation of information that was not anticipated at design time [20]. Fixed roles and fixed channels therefore fail together, and the coupling is structural rather than incidental.

Dynamic role regimes respond by making the organizational structure itself an object of optimization. Evolving orchestration treats the assignment of agents to roles as a policy that is revised as execution proceeds, allowing the team to reallocate capacity toward subtasks that turn out to be difficult [91]. Decentralized evolutionary coordination pushes the same idea further by removing the central allocator entirely: agents negotiate and revise their own responsibilities through local interaction, so that the role structure is a product of the coordination process rather than an input to it [89]. The mechanism common to both is feedback-driven reallocation, and the difference between them is where the reallocation decision is made—centrally, on the basis of global state, or locally, on the basis of neighbor state.

A third family treats role evolution as a resource-allocation problem rather than a competence problem. Dynamic agent elimination prunes agents whose contributions do not justify their token cost, effectively shrinking the role set during execution [90]. This reframes the fixed-versus-dynamic question: the objective is not to discover better roles but to discard redundant ones, and the reported gains are correspondingly concentrated in token efficiency rather than in accuracy on tasks that the original team could already solve. The distinction matters for interpretation, because elimination improves the cost-accuracy frontier by moving along it, whereas orchestration evolution attempts to shift the frontier by changing what the team can do.

Scale introduces a further axis on which the two regimes diverge. Cross-team orchestration organizes collaboration among multiple teams rather than multiple individuals, and at this granularity the role structure must be hierarchical: teams occupy stable functional positions while membership within teams remains fluid. Evidence from scaling studies of LLM-based collaboration indicates that performance does not improve monotonically with team size, which suggests that role proliferation beyond a threshold adds coordination overhead faster than it adds capability [93]. Fixed roles mitigate this overhead by making interaction patterns predictable; evolutionary roles risk amplifying it, since every reallocation is itself a coordination event.

The tension is sharpest where reward must be attributed to individual roles. Reward attribution and decomposition methods attempt to assign credit for team outcomes to the agents responsible, and this assignment is straightforward when roles are stable and ambiguous when they are not [94]. A dynamically evolving role set means that the mapping from contribution to credit changes over the course of a single episode, which weakens the learning signal that role-evolution mechanisms depend on. This is a genuine coupling between two design choices that are usually discussed separately: the more adaptive the role regime, the more difficult the credit assignment that sustains it.

Comparative reading of the evidence therefore yields a conditional rather than a universal recommendation. Fixed roles dominate when tasks are standardized, when auditability is required, and when team size is large enough that coordination overhead is the binding constraint [92, 51]. Evolutionary roles dominate when the task distribution is open-ended and when the cost of a missing competence exceeds the cost of renegotiation [91, 89]. Elimination-based approaches occupy an intermediate position, delivering cost reductions without altering the competence set [90]. Surveys of the field's business and technical perspectives reach a compatible conclusion, framing role design as a trade-off surface rather than a best practice [78].

Two boundary conditions qualify this picture. First, the reported benefits of role evolution are measured predominantly on benchmarks where the initial role set is deliberately underspecified; whether evolution retains its advantage when the seed roles are well designed remains untested, and the scaling evidence hints that the margin may narrow [93]. Second, decentralized evolution depends on local interaction quality, and communication surveys document that sparse or noisy channels degrade exactly the neighbor information on which local reallocation relies [20]. The failure modes of the two regimes are thus not symmetric: fixed roles fail by omission, evolutionary roles fail by oscillation.

These failure modes can couple. When an evolutionary mechanism reallocates roles in response to a locally observed failure, and the underlying cause is a communication deficit rather than a competence deficit, the reallocation addresses the symptom and leaves the cause intact, producing repeated reorganization without improvement [7]. Elimination mechanisms are exposed to the same confusion in reverse: an agent may appear redundant because its channel is degraded rather than because its competence is superfluous [90]. Distinguishing structural from informational causes of poor performance is therefore a precondition for any role-adaptation policy, and current designs largely leave this diagnosis implicit.

Taken together, the evidence supports a layered view in which role regime, communication structure, and credit assignment are jointly determined rather than independently selectable. Fixed roles are economical but brittle; evolutionary roles are adaptive but expensive to sustain and hard to credit; elimination is cheap but does not expand competence. The unresolved question is whether these regimes can be composed—stable functional roles with fluid membership, or centralized structure with decentralized reallocation—and answering it requires understanding how the orchestration topology itself constrains what any role mechanism can achieve, which is where the analysis turns next.

The design space therefore determines what the evaluation layer can attribute to collaboration: topology and role choices fix which quantities are observable, and the evaluation chapter asks whether existing benchmarks measure them. In the reverse direction, evaluation results recalibrate this design space: a topology that wins on saturated HumanEval need not win on BIRD.

7 FRAGMENTATION OF EVALUATION BENCHMARKS AND THE NEED FOR CROSS-TASK UNIFICATION

Since the realizable space of role mechanisms is constrained by the orchestration topology itself, clarifying how topological structure delimits collaboration forms and information flow becomes a prerequisite for evaluating the feasibility of various role arrangements.

Having established how collaboration architectures and role division trade off under centralized and decentralized designs, the review now turns to the measurement layer that is meant to adjudicate those trade-offs. Current benchmarks cover task capability narrowly and deeply: they saturate on single-agent, single-turn, auto-gradable tasks while leaving multi-agent collaboration, long-horizon planning, safety, and high-stakes domains largely unmeasured, and the metrics they report are systematically misaligned, with the same names denoting different quantities and different names denoting the same one. This chapter therefore examines benchmark coverage and its blind spots, traces the semantic drift of evaluation metrics, and derives the dimensions along which a unified cross-task framework could be organized. That framework, however, measures capability without measuring what happens when capability is attacked, which leads directly into the adversarial robustness gaps at safety intersections taken up next.

7.1 Task-Coverage Profiles and Blind Spots of Mainstream Benchmarks

A central question for any cross-task unification effort is what the current mainstream benchmarks—HumanEval, MBPP, MMLU, GSM8K, MATH, Spider, and BIRD—actually measure, and whether their combined coverage leaves systematic gaps. The evidence reviewed here indicates that these benchmarks were each designed around a single capability axis, and that their aggregation produces a coverage profile that is broad in surface task variety but thin in interaction, tool use, and failure-mode coverage.

Code-generation benchmarks anchor the most mature part of this field. HumanEval and MBPP evaluate functional correctness of short Python programs against unit tests, and they have become the default yardstick for reasoning-to-code transfer in agent-oriented reviews [4]. Their strength is verifiability: a generated program either passes the tests or does not, which yields a low-noise signal. However, both benchmarks are single-function, single-file, and specification-complete; they do not require a system to decompose a task, consult external resources, or coordinate multiple components. Consequently, strong HumanEval performance is compatible with weak performance on the multi-step, tool-mediated tasks that agentic systems are built for, a discrepancy that the agent literature repeatedly observes [3].

Knowledge and mathematical reasoning benchmarks occupy a different axis. MMLU probes broad factual and disciplinary knowledge through multiple-choice items, while GSM8K and MATH probe multi-step arithmetic and symbolic derivation. These benchmarks are effective at isolating parametric knowledge and chain-of-thought competence, and reviews of LLM reasoning treat them as standard reference points [4]. The limitation is structural rather than incidental: all three are closed-world and single-turn. The answer is fully determined by the prompt, no external state is consulted, and no intermediate artifact must remain consistent across steps. This design choice makes scores comparable across models but renders the

benchmarks silent on whether a system can maintain correctness when information must be retrieved, verified, or revised.

Database-query benchmarks introduce the first genuinely interactive axis. Spider and BIRD evaluate natural-language-to-SQL translation, with BIRD adding external evidence and emphasizing schema-aware generation over realistic databases. Work on agentic NL2SQL shows that schema awareness and iterative clarification materially change performance, and that systems which decompose the query and inspect the schema outperform single-pass translation. This is an important contrast with HumanEval-style evaluation: here the environment (schema, evidence, database contents) is part of the problem, and correctness depends on how the system acquires and uses that context. Yet Spider and BIRD still bound the interaction to a single database and a single query, so they do not test multi-tool orchestration or cross-source consistency.

A first blind spot therefore emerges at the level of coordination. None of the seven benchmarks requires multiple agents, roles, or delegated subtasks; they score a single model's output on a single input. Yet multi-agent frameworks are evaluated almost exclusively by transplanting these single-agent benchmarks onto collaborative architectures. MetaGPT is assessed on code-generation and software-engineering tasks [14], and collaboration-oriented planning work reports gains on task-planning suites [95]. The mismatch is that the benchmark cannot distinguish whether an observed gain comes from collaboration or from the underlying model, because the benchmark was never designed to isolate the contribution of coordination.

A second blind spot concerns tool use under realistic protocols. FinMCP-Bench targets financial tool use under the Model Context Protocol and exposes failures that knowledge-and-code benchmarks cannot surface, including incorrect tool selection and malformed invocation sequences. This contrasts sharply with MMLU and GSM8K, where the only failure mode is a wrong answer. In tool-mediated settings, failure can be a wrong answer, a wrong tool, a correct tool invoked with wrong arguments, or a correct invocation whose result is misinterpreted—a taxonomy that closed-world benchmarks collapse into a single incorrect label.

A third blind spot is adversarial and safety behavior. TAMAS benchmarks adversarial risks in multi-agent LLM systems and shows that systems which score well on capability benchmarks can nonetheless be manipulated through injected messages or compromised peers [56]. Related work on constraint drift argues that safe behavior must be continuously maintained rather than asserted once at specification time [2]. Neither HumanEval nor MMLU nor Spider contains any adversarial condition, so a model's capability score carries no information about its robustness. The two properties are measured on disjoint instruments, and no mainstream benchmark reports them jointly.

A fourth blind spot is domain specialization. Legal assistance, electronic health record modeling, and financial analysis each impose domain-specific correctness criteria and regulatory constraints that generic benchmarks do not encode. Chatlaw illustrates a role-aligned mixture-of-experts design for legal assistance [16], and ColaCare shows that EHR modeling benefits from LLM-driven multi-agent collaboration [86]. In both cases the evaluation must be domain-native; a high MMLU score says little about whether a system produces a defensible legal citation or a clinically valid record. The coverage gap here is not a missing task type but a missing validity criterion.

A fifth gap concerns traceability and accountability. Role-specialized multi-agent pipelines raise questions about which component produced which claim, and traceability work argues that accountability must be designed into the pipeline rather than reconstructed afterward [8]. Mainstream benchmarks score only final outputs, so they cannot reward or penalize the provenance of intermediate steps. This is a qualitative difference from, say, BIRD, where the generated SQL is itself the artifact under inspection; in multi-agent settings the artifact is a trajectory, and no benchmark in the reviewed set scores trajectories directly.

Notably, the fragmentation is not merely a matter of missing categories but of incompatible measurement semantics. Code benchmarks report pass rates, knowledge benchmarks report accuracy, and NL2SQL benchmarks report execution accuracy or exact match; these quantities are not commensurable, and averaging them into a composite score would obscure precisely the trade-offs that matter. Reviews of trust, risk, and security management in agentic systems make the same point from a different angle: evaluation must cover capability, safety, and governance jointly, because optimizing one can degrade another. The benchmark field thus exhibits both coverage gaps and metric incommensurability, and the two problems compound.

The practical consequence is that claims of general competence are underdetermined by the available instruments. A system can report strong HumanEval, MMLU, and GSM8K numbers while remaining untested on tool selection, adversarial robustness, domain validity, and trajectory-level accountability. Systematic reviews of embodied and generative multi-agent collaboration reach a similar conclusion, noting that evaluation practice lags behind architectural ambition. This does not mean the existing benchmarks

are flawed; it means they were built to answer narrower questions than the ones now being asked of them.

Taken together, the coverage profile is complementary but incomplete: code, knowledge, mathematics, and database querying are well instrumented, whereas coordination, tool use, adversarial robustness, domain validity, and traceability are either unmeasured or measured only in specialized, non-comparable settings. This asymmetry motivates the next step of the chapter, which examines how evaluation metrics themselves diverge across these benchmark families and why cross-task comparability cannot be recovered by simple aggregation.

7.2 Semantic (Mis)Alignment of Evaluation Metrics Across Benchmark Families

Do the metrics that recur across multi-agent benchmarks actually denote the same quantity, or has a shared vocabulary masked divergent operational definitions? This section answers that question by examining six frequently reported metrics—Accuracy, Success rate, Pass@1, Completeness, Executability, and Efficiency—and finds that their semantic definitions are systematically unaligned, with same-name-different-meaning and synonymy coexisting as twin pathologies.

The most consequential defect is same-name-different-meaning, and Success rate is its clearest instance. The label is reused pervasively, yet its operationalization drifts across three mutually exclusive semantics. In one strand it denotes exact string matching between an output and a ground-truth answer; in another it denotes the percentage of generated federated-analytics operations that are executable and correctly ordered, effectively a completion rate over the execution DAG, reported at 95–100% against 10–15% for zero-shot baselines [51]. In robotic collaboration, by contrast, a task is failed if agents collide, interfere, or time out, and one study refines this into five distinct failure conditions including token overflow and consensus failure. These three definitions are not refinements of one another; they measure exact-match correctness, structural executability, and physical constraint satisfaction respectively, and no monotone mapping connects them.

Executability exposes an even deeper incompatibility because its divergence crosses measurement levels rather than merely differing in referent. One framework defines it as the percentage of successfully executable modified architectures—a binary proportion [42]—whereas another rates code on a four-point ordinal scale from failure to flawless, and a third reports the percentage of software that compiles without errors. A proportion, an ordinal rating, and a compile-pass percentage differ in scale type, not just in threshold, so even rank-order comparisons across papers are unsafe. Accuracy exhibits the same pattern at higher density: its definitions span entity-linking correctness, categorical classification accuracy, binary response correctness, and numerical error against a guessed value, each attached to a different task family. The label has become a generic placeholder for "how right the output was," with the operative notion of rightness left to each paper.

The mirror-image pathology, synonymy, shows that identical underlying events are reported under different names, which inflates apparent metric diversity while concealing real agreement. In code generation, Pass@1, Success rate, Generation accuracy, Executability, and Pass@5 all measure whether a generated artifact passes syntax and functional tests; one study states explicitly that a run counts as a success only when it passes both syntax and functional tests, which coincides with the compile-pass notion of Executability used elsewhere [14]. RTL and chiplet design benchmarks report Pass@1 for simple design success and Pass@5 for realistic chiplet generation, yet the former is elsewhere called generation accuracy and elsewhere success rate [30, 31]. Synonymy also manifests as conflicting decomposition schemes for software quality: a four-element tuple of Completeness, Executability, Consistency, and Quality shares only Executability with a five-element tuple of Executability, Cost, Code Statistics, Productivity, and Human Revision Cost, leaving "software quality" partitioned in two non-overlapping ways [14].

Efficiency illustrates how synonymy and dimensional heterogeneity compound. Validation efficiency is reported as a composite of correctness and execution speed at 51.05% [39], operation count is an absolute tally over six federated-analytics operation types, cost per prompt is denominated in dollars, and transmission overhead is measured in bytes [51]. These four quantities belong to ratio, count, currency, and data-volume dimensions respectively and cannot share a coordinate axis. A three-dimensional Pareto frontier over accuracy, cost per prompt, and latency is therefore not comparable to a scalar composite, even when both are described as measuring efficiency [39]. The consequence is that reported efficiency figures are commensurable only within the paper that produced them. Table 4 collects the resulting label–operationalization mismatches.

These definitional divergences carry direct quantitative consequences for cross-task comparability. Three studies report success-rate figures that appear comparable in magnitude but are measured over different task suites and failure criteria—exact-match task success, DAG structural validity, and physical

Table 4: Metric labels versus operationalizations. Identical labels denote different quantities and scale types, so cross-paper rankings are unsafe even when the reported numbers look comparable.

Label	Operationalizations in the evidence	Scale type	Consequence
Success rate	Exact-match task success; DAG structural validity; physical or dialogue completion [51, 96]	Proportion	Rankings change with the failure criterion.
Executability	Percentage of executable architectures [42]; four-point ordinal rating; compile-pass percentage	Proportion, ordinal	Rank-order comparisons across papers are unsafe.
Accuracy	Entity-linking correctness; classification accuracy; binary response correctness; numerical error	Mixed	The label becomes a placeholder for "how right".
Efficiency	Composite correctness–speed score (51.05%); operation count; cost per prompt; bytes transmitted [51, 39]	Ratio, count, currency, volume	No shared coordinate axis; comparisons hold only within a paper.

or dialogue completion respectively [51, 96]; read side by side they appear to describe the same quantity. Aggregating such numbers would produce a statistic with no defined referent. The drift is not merely terminological: because the definitions differ in what counts as failure, they induce different orderings over systems, so a method that leads on one operationalization may trail on another.

The literature itself acknowledges this as a structural deficiency rather than an incidental oversight. A comprehensive review states directly that the field of evaluation benchmarks, frameworks, and collaboration protocols for LLM-based agents is fragmented and lacks a unified taxonomy [4]. A second line of work locates the ambiguity of "success" in the unresolved choice between user-side and system-side perspectives, since users may not know what happens behind the scenes, and responds by proposing four distinct Goal Success Rate variants—overall, supervisor, user-side, and system-side—validated by human annotation agreement of 0.87–1.00 [5]. That validation is instructive: explicit operationalization appears sufficient to restore comparability without changing the underlying task.

A third strand warns against the scalarization that typically accompanies metric proliferation. A five-dimensional assessment framework spanning trustworthiness, explainability, user-centeredness, coordination, and composite performance cautions that scalarization can hide critical failures and recommends reporting the full metric vector rather than a single weighted score. Embodied-AI reviews reach a compatible conclusion from a different direction, observing that existing benchmarks lack standardized evaluation criteria and concentrate on task-specific metrics. Read together, these reviews converge on the diagnosis that the missing element is a shared taxonomy of what each metric measures, not a shortage of metrics.

The divergence is not confined to the six headline metrics; it recurs wherever multi-agent systems are evaluated on domain-specific outcomes. Legal-assistant evaluations, clinical decision-support evaluations, and financial tool-use benchmarks each define task success against domain-specific validity criteria that have no counterpart in the general benchmarks, so the same nominal metric carries an additional layer of domain semantics [16, 86, 26]. This widens the gap rather than narrowing it: cross-domain comparison requires aligning not only the metric's operational definition but also the domain's criterion of correctness.

Two boundary conditions qualify these findings. First, the evidence base, while covering all six metrics, is drawn from a bounded set of papers, so more consistent definitions may exist in unsampled work; conclusions about Completeness in particular rest on comparatively thin support and should be held with corresponding caution. Second, the desirability of quantitative unification is itself contested: some evaluations deliberately adopt qualitative assessment rather than constructing a benchmark around quantitative metrics, a stance that conflicts with the push toward a unified quantitative framework and signals methodological resistance to any alignment proposal. Alignment efforts must therefore argue not only that metrics can be harmonized but that harmonization is worth its cost.

The root cause can be stated compactly: evaluation practice lacks a clean decomposition of what is measured from how it is operationalized. The disambiguation attempts reviewed above point toward a remedy—separating the measurement object (artifact correctness, process validity, resource cost) from the measurement scale (proportion, ordinal, count, currency) as two independent dimensions, and requiring each reported metric to register its operational definition explicitly. Whether such registration can be enforced, and how divergent definitions might be normalized automatically, is the subject of the following section.

7.3 Dimensions of a Unified Evaluation Framework and Reusable Components in Existing Work

What dimensions must a unified evaluation framework span, and which reusable components do existing systems already supply? Four dimensions recur across the literature: task type, collaboration scale, error tolerance, and cost. The five-dimensional and three-dimensional frameworks discussed in this section are prior proposals, not alternative counts for the four-dimension scheme used here. These are not independent axes but coupled constraints, and the evidence suggests that current benchmarks and frameworks each cover only a subset, leaving the intersections largely unmeasured.

Task type is the most developed dimension, yet coverage is uneven. Domain-specific benchmarks anchor task type to concrete tool ecologies: FinMCP-Bench evaluates financial tool use under the Model Context Protocol and reports that agent performance degrades sharply as the number of available tools grows [26], while schema-aware NL2SQL systems show that task structure—here, database schema—must be encoded into the agent loop rather than assumed [39]. General reviews, in contrast, organize task type taxonomically across reasoning, planning, and acting without committing to a measurement instrument [4]. The contrast is instructive: domain benchmarks supply executable task definitions but narrow scope, whereas reviews supply breadth but no operational metric. A unified framework must therefore borrow task instantiations from the former and the taxonomic scaffolding from the latter.

Collaboration scale is the dimension on which existing work diverges most sharply, and the divergence is architectural rather than merely quantitative. Centralized orchestration, exemplified by MetaGPT's role-based assembly line, encodes standard operating procedures so that a fixed team of agents can be evaluated against a stable workflow [14]. Scaling studies show that such architectures improve with team size up to a point before communication overhead dominates [93], and cross-team orchestration extends the unit of analysis from the team to the coalition [97]. Decentralized designs, by contrast, trade coordination guarantees for robustness, a trade-off made explicit in multi-robot settings where centralized and decentralized systems are compared under identical task loads [96]. The implication is that scale cannot be reported as a single number; it must be registered jointly with the coordination topology under which it was measured.

Error tolerance constitutes the weakest-covered dimension and the one most often conflated with average-case accuracy. Failure analyses of autonomous agents identify recurring breakdowns in planning, tool invocation, and self-verification [3], and adversarial benchmarking shows that multi-agent systems can be driven into unsafe states through injected messages even when each individual agent passes its own checks [56]. Constraint drift offers a mechanistic account: safety constraints asserted at initialization decay over successive interaction rounds, so tolerance is a function of horizon rather than of any single step [2]. Temporal graph monitoring approaches attempt to detect such drift online [82], but detection is not the same as a tolerance specification. A unified framework thus needs an explicit fault-injection protocol and a horizon-dependent tolerance threshold, neither of which is standard today.

Cost is the dimension where reusable components are most mature, yet they are rarely reported alongside effectiveness. OPTIMA reports that communication dominates token expenditure in LLM-based multi-agent systems and demonstrates that aggressive message compression preserves task effectiveness at a fraction of the cost. Efficiency-oriented protocol design in adjacent areas confirms that communication volume can be reduced without sacrificing correctness when the protocol is restructured rather than merely truncated. The recurring pattern is that effectiveness and cost are measured in separate papers on separate setups, which makes the Pareto frontier across studies incomparable. Cost therefore belongs in the same report as effectiveness, expressed in tokens, wall-clock time, and monetary units jointly.

Beyond the four primary dimensions, trust and accountability supply cross-cutting requirements that any unified framework must accommodate. Reviews of trust, risk, and security management for agentic systems argue that evaluation must cover not only outcomes but the provenance of decisions [80], and traceability work in role-specialized pipelines shows that per-role attribution is technically feasible when the pipeline records intermediate artifacts [8]. Specification-level evaluation prototypes in cybersecurity demonstrate that formal properties can be checked against running multi-agent systems [5], which suggests that trust requirements are amenable to the same instrumentation as performance metrics rather than being relegated to qualitative discussion.

Reusable components for constructing such a framework already exist in partial form. Role-aligned mixture-of-experts architectures provide a template for decomposing a task into evaluable sub-roles [16], and clinical multi-agent systems show how domain experts can be instantiated as agents whose outputs are individually auditable [86]. Planning frameworks that combine chain-of-thought with multi-agent collaboration offer a reusable task-decomposition harness [95], while reinforcement-learning-based planners paired with graph policies supply a reusable policy-evaluation substrate [98]. Neural architecture design

via multi-agent collaboration further shows that the search process itself can be instrumented as a sequence of evaluable decisions [42]. Each component addresses one dimension well; none spans all four.

The gap is therefore one of integration rather than invention. Embodied-AI reviews catalogue collaboration patterns but do not prescribe measurement [12], and general surveys of intelligent generative agents describe capabilities without a common metric vocabulary. Federated analytics over decentralized sources illustrates the further complication that data cannot be centralized for evaluation, so cost and privacy must be scored together [51]. These observations indicate that the four dimensions are not additive: a framework that fixes task type and scale but ignores tolerance and cost will systematically overstate deployability.

Taken together, the literature supplies the ingredients of a unified framework—task instantiations from domain benchmarks, topology-aware scale measures, fault-injection and drift-detection mechanisms, and communication-cost accounting—but no work assembles them into a single instrument. The dimensions are coupled, and the coupling is precisely what current evaluations omit. This motivates the next section's concern: how such a framework can be specified so that results remain comparable across tasks, scales, and failure regimes.

Metric semantics therefore define what counts as safety evidence: a benchmark that cannot express an adversarial condition cannot certify a defense, and the safety chapter traces where that gap appears. In the reverse direction, safety failures expose metric gaps, so the four dimensions above are revised by what the protection layer cannot certify.

8 ADVERSARIAL ROBUSTNESS GAPS IN SAFETY INTERSECTIONS

Building on the preceding call for cross-task unification, this chapter examines the protection layer of the governance stack, where adversarial safety and federated constraints are meant to secure multi-agent LLM collaboration. The central tension is that the threats named in the literature and the constraints inherited from federated learning are treated as neutral engineering conventions, yet they map onto collaborative settings as structural assumptions that quietly reshape privacy and decentralization. The discussion proceeds from explicitly named adversarial risks and attacker capabilities, through the bounded-gradient, client-aided-model, and delay-free-communication constraints, to the composability of robust aggregation, adversarial detection, and privacy constraints—revealing local defenses that fail to cohere into an end-to-end safeguard. This unresolved protective gap ultimately redirects attention to the coordination layer, where communication constraints govern how information is shared and defended.

8.1 Explicit Threat Naming in Multi-Agent LLM Systems: From Communication Attacks to Context Poisoning and Behavioral Drift

What adversarial and security threats have been explicitly named for multi-agent LLM systems? The literature converges on a layered threat picture in which attacks are classified by the surface they exploit—inter-agent messages, external tool and context channels, and the slow erosion of behavioral constraints over time—rather than by a single adversarial objective. This section organizes the named threats along these three primary surfaces, and treats covert signaling and accountability ambiguity as two further named threats that do not fit the triad. It then contrasts how different benchmarks and attack frameworks characterize attacker capabilities and required background knowledge.

At the communication layer, the named threat is the manipulation of inter-agent messages to subvert collective decisions. Red-teaming work formalizes communication attacks as a distinct class in which an adversary injects or alters the content exchanged between agents, showing that such interference can redirect the system's output without touching model weights or the task specification [99]. Earlier adversarial studies on multi-agent communication reach a compatible conclusion from a different angle: message channels are a viable attack surface precisely because downstream agents treat incoming messages as trusted evidence [55]. The two lines differ in emphasis—one frames the attacker as an external red-teamer probing a deployed system, the other as a perturbation process acting on the communication graph—yet both identify the same mechanism, namely that trust in inter-agent content is assumed rather than verified. This convergence suggests that communication integrity, not model alignment alone, is the operative defense boundary.

A second named threat concerns the context and tooling substrate that agents consume, particularly through the Model Context Protocol. Systematic security analysis of MCP identifies a family of risks arising from the protocol's role as a mediating layer between agents and external resources, including the potential for malicious or compromised servers to shape what an agent perceives as authoritative context [100]. Poisoning benchmarks sharpen this into a concrete attacker capability: an adversary who controls

the documentation or manual an agent relies on can induce unsafe actions even when the underlying model behaves correctly on clean inputs [101]. Compared with communication attacks, context poisoning demands less access to the runtime message stream but more background knowledge about the agent's retrieval and tool-use habits; the benchmark evidence indicates that realistic, manual-level poisoning is sufficient to degrade agent behavior, which lowers the assumed attacker sophistication relative to gradient- or weight-level threats.

Benchmarking efforts consolidate these surfaces into a structured taxonomy of adversarial risks. TAMAS proposes a systematic categorization of risks specific to multi-agent LLM systems and evaluates agents against them, establishing that distinct risk categories produce distinct failure signatures rather than a uniform degradation [56]. This matters for threat naming because it implies that defenses calibrated to one category—say, prompt injection into a single agent—do not transfer automatically to multi-agent coordination failures. In contrast to single-agent robustness benchmarks, which typically vary input perturbations along one axis, the multi-agent setting multiplies the axes: which agent is compromised, at what stage, and through which channel. The taxonomy therefore functions less as a checklist and more as a claim about the dimensionality of the attack space.

Beyond instantaneous attacks, a third named threat is the gradual degradation of system behavior over extended interaction, often termed drift. Empirical work quantifies behavioral degradation in multi-agent systems across long horizons, showing that performance and adherence to intended behavior decay as interactions accumulate rather than failing abruptly [1]. Constraint-drift research extends this by arguing that safe multi-agent behavior must be actively maintained rather than asserted once at design time, documenting how constraints erode through successive agent handoffs [2]. These two findings are complementary: one measures the phenomenon, the other locates its mechanism in the absence of ongoing constraint enforcement. The implication is that drift is not an adversarial attack in the classical sense—no attacker is required—but it belongs in the threat inventory because it produces the same outcome an attacker would seek, and because an adversary can deliberately accelerate it.

Covert channels constitute a further explicitly named risk that sits between communication attacks and drift. Event-driven covert communication protocols demonstrate that agents can exchange information through side channels that evade content-level inspection, raising the prospect of collusion or data exfiltration that standard message monitoring would miss [102]. This threat is notable for its low detectability rather than its potency: the volume of covert signaling may be small, but its presence undermines the assumption that observable inter-agent traffic is a complete account of inter-agent influence. Relative to red-teaming communication attacks, which assume the adversary perturbs visible messages, covert-channel work assumes the adversary exploits the timing and event structure of the system—a different capability profile with different background knowledge requirements.

Accountability and traceability concerns complete the named threat set by addressing what happens after a failure. Work on role-specialized multi-agent pipelines shows that when responsibility for an action is distributed across specialized roles, attributing an adverse outcome to a specific agent or decision becomes difficult, which weakens both detection and remediation [8]. This is a threat in the security sense because an adversary who can exploit role boundaries benefits from the resulting ambiguity; it is also a structural vulnerability independent of any attacker. Survey-level synthesis of trust, risk, and security management for agentic systems places these concerns within a broader governance frame, noting that trust assumptions are rarely made explicit and therefore rarely audited [80]. The contrast with the attack-centric literature is instructive: attack papers name capabilities, while governance work names the organizational conditions under which those capabilities go unnoticed.

The named threats also inherit classical adversarial and cryptographic concerns, though the mapping is partial. Gradient-based leakage attacks establish that shared artifacts in distributed training can reveal private inputs [103], a risk that reappears when multi-agent systems share intermediate representations or memory. Secure two-party protocols offer a constructive counterpoint, showing that communication-efficient cryptographic techniques can in principle protect exchanged values. Juxtaposing the two reveals a gap: the attack literature demonstrates leakage from shared state, and the cryptographic literature demonstrates protection for well-specified two-party computations, but neither addresses the many-party, loosely specified, natural-language communication that characterizes LLM agent systems. This mismatch is where several of the named threats remain least understood.

Taken together, the explicitly named threats fall into a coherent inventory: communication manipulation, context and tool poisoning, covert signaling, behavioral and constraint drift, and accountability ambiguity, with classical leakage and secure-computation concerns providing adjacent reference points. The categories differ systematically in attacker capability and background knowledge—communication attacks require runtime access, poisoning requires control of external content, covert channels require structural

insight, and drift requires no attacker at all. This variation in assumptions is itself a finding: comparisons across benchmarks are only meaningful when the assumed capability profile is stated. The next section examines where these named threats intersect with safety mechanisms, and why the intersections remain weakly defended. Table 5 summarizes this inventory.

The five surfaces can be stated compactly: (i) communication manipulation, which targets inter-agent messages and requires runtime access [99, 55]; (ii) context and tool poisoning, which targets external manuals and MCP servers and requires control of that content [100, 101]; (iii) covert signaling, which exploits timing and event structure and requires structural insight [102]; (iv) behavioral and constraint drift, which emerges over long-horizon interaction and requires no attacker at all [1, 2]; and (v) accountability ambiguity, which exploits distributed role boundaries [8, 80]. This list is the prose counterpart of Table 5, and it is the inventory that the composability analysis below draws on.

Table 5: Five named threat surfaces, the attacker capability each requires, representative evidence, and the state of defenses.

Threat surface	Attacker capability	Evidence	Defense status
Communication manipulation	Runtime access to inter-agent messages	[99, 55]	Integrity checks at protocol boundaries; no end-to-end guarantee
Context and tool poisoning	Control of external manuals, tools, or MCP servers	[100, 101]	Sanitization at ingestion; weak across handoffs
Covert signaling	Structural insight into timing and event structure	[102]	Low detectability; content monitoring insufficient
Behavioral and constraint drift	No attacker required	[1, 2]	Periodic re-anchoring; evidence limited to few domains
Accountability ambiguity	Exploitation of distributed role boundaries	[8, 80, 54]	Provenance tooling early; trust assumptions seldom audited

8.2 Mapping from Federated Learning Constraints to Multi-Agent Privacy and Decentralized Training

How do the standard constraints of federated learning, such as bounded gradients, client-aided computation, and delay-free communication, carry over to privacy-preserving and decentralized training in multi-agent systems? The short answer is that they carry over only partially: the cryptographic and optimization assumptions that make federated learning tractable are precisely the assumptions that adversarial multi-agent research has learned to violate. The literature supplies two largely separate bodies of evidence. One body establishes what client-aided secure computation can guarantee under communication and trust constraints [104], and what gradient-level leakage reveals about the fragility of the bounded-gradient premise. The other body documents how communication channels, context protocols, and long-horizon interaction become attack surfaces in LLM-based multi-agent systems [55]. Reading the two together exposes where the federated abstraction and the agentic reality diverge.

The bounded-gradient assumption is the weakest point of translation, because it is a statement about the optimizer that adversaries can simply decline to honor. Gradient-inversion work shows that shared updates, even when aggregated, retain enough information to reconstruct private training inputs with high fidelity [103]. In federated learning this risk is contained by clipping, noise injection, and a semi-honest server model; in a decentralized multi-agent setting, however, the analogue of a shared gradient is a shared message or memory state, and no clipping norm bounds what an agent may encode in natural language. The mechanism of leakage therefore shifts from numerical inversion to semantic disclosure, and the defenses that rely on bounding update magnitude do not transfer. This suggests that privacy guarantees formulated over parameter vectors must be re-derived over communication transcripts before they can be claimed for agentic training.

Client-aided protocols offer the most direct constructive mapping, and they also delimit its cost. Communication-efficient client-aided two-party protocols show that outsourcing expensive cryptographic operations to a semi-trusted client can reduce round complexity substantially while preserving security against a corrupted party [104]. The structural correspondence with multi-agent collaboration is close: a coordinating agent plays the client-aided role, and specialized agents hold partitioned inputs. Yet the security argument of such protocols presumes a fixed number of parties, a defined function, and an adversary who cannot alter the protocol specification. Multi-agent systems violate all three, since agents are added

dynamically, their roles are specified in natural language, and the specification itself is an attack surface. In contrast to the cryptographic setting, where deviation from the protocol is detected by the protocol, agentic deviation is often indistinguishable from legitimate role behavior.

The delay-free communication assumption is the one that fails most visibly, and its failure is coupled with security rather than merely with performance. Federated optimization tolerates asynchrony through staleness bounds; the agentic analogue has no such bound, because messages are interpreted rather than averaged. Red-teaming studies demonstrate that manipulating inter-agent communication, rather than compromising any single model, is sufficient to redirect system behavior [99], and earlier work on multi-agent communication attacks establishes that the channel itself is a first-class attack surface [55]. A delay-free model implicitly assumes that a message arrives unmodified and is read as intended; both assumptions are empirically false. The consequence is that robustness cannot be localized to individual agents, which is the default posture of most federated defenses.

Decentralization introduces a further divergence: the trust model. Federated learning typically assumes an honest-but-curious server or a threshold of honest clients, and it reasons about privacy against a defined coalition size. Multi-agent benchmarks instead enumerate concrete adversarial capability profiles, and their findings are conditional on those profiles. TAMAS structures adversarial risk evaluation around explicit threat categories spanning input, inter-agent, and system-level manipulation [56], while protocol-level toolkits examine how context and tool interfaces expose agents to injection and exfiltration. These are not refinements of a single threat model; they are different threat models. Comparisons between a federated privacy bound and an agentic red-teaming result are therefore meaningful only when the assumed adversary is stated on both sides.

The context and tool layer has no federated counterpart at all, and this is a genuine coverage gap rather than a difference of emphasis. In federated learning the interface between parties is a numerically specified update; in agentic systems the interface is a manual, a schema, or a set of tool descriptions. Poisoning benchmarks show that falsified documentation alone can induce unsafe agent behavior without any parameter-level compromise. This failure mode is invisible to gradient-based privacy analysis, because no gradient is involved. The implication is that the attack surface of decentralized training expands from the update channel to the entire specification layer, and defenses designed for the former leave the latter unprotected.

Long-horizon interaction adds a temporal dimension that federated analyses do not model. Federated privacy is usually argued per round or per training run, with composition handled by accounting. Agentic systems degrade behaviorally over extended interaction, a phenomenon quantified as drift and attributed to accumulating context rather than to any single malicious input [1]. Constraint drift extends this observation to safety properties, showing that constraints asserted at deployment are not maintained through sustained interaction [2]. The parallel with privacy budget composition is instructive but imperfect: privacy loss composes predictably under stated assumptions, whereas behavioral drift appears to compose unpredictably and without an adversary. This indicates that decentralized training protocols need maintenance mechanisms, not only accounting mechanisms.

Accountability structures provide a partial bridge between the two literatures, and their limits are informative. Role-specialized pipelines can be instrumented so that each agent's contribution to an outcome is traceable, which is the operational analogue of attributing a leaked update to a participating client. Traceability, however, is a detective control; it establishes what happened after the fact and does not prevent disclosure. The federated literature's preventive controls, namely secure aggregation and noise, have no established agentic equivalent. A natural question is whether cryptographic aggregation can be applied to agent messages at all, given that messages must remain interpretable to be useful. Current evidence does not answer this, and the gap is structural rather than incidental.

Covert channels sharpen the contrast between assumed and actual communication semantics. Event-driven covert communication protocols demonstrate that agents can exchange information through timing and event patterns that are invisible to content-level inspection [102]. A delay-free, content-inspected channel model cannot represent such leakage, and neither can a federated model that treats communication as a vector of bounded norm. The mechanism is fundamentally different from gradient inversion: nothing is reconstructed, and no parameter is exposed, yet information crosses a boundary that the system designer believed closed. This suggests that privacy in decentralized agent systems must be defined over observable behavior, not only over transmitted content.

Systematic trust, risk, and security reviews of agentic multi-agent systems converge on a layered framing in which governance, identity, and communication each require separate treatment [80]. That framing is useful here because it explains why federated constraints do not map one-to-one: federated learning is a single-layer discipline that assumes the other layers are stable. When identity, specification, and

interaction horizon are all in play, a constraint such as bounded gradients addresses only one of several disclosure paths. The layered view also clarifies which federated results remain portable: those concerning the update channel transfer with modification, while those concerning party behavior and specification integrity do not.

Taken together, the evidence supports a qualified conclusion. Client-aided secure computation transfers to decentralized agent collaboration as a design pattern for partitioning sensitive computation, and its communication-efficiency arguments remain relevant [104]. The bounded-gradient premise does not transfer, because agentic leakage operates through semantics and timing rather than through parameter reconstruction [103, 102]. The delay-free assumption fails in a way that couples performance and security, since the channel is itself an attack surface [99]. Decentralized training for multi-agent systems therefore requires privacy definitions stated over transcripts and behavior, together with maintenance guarantees across interaction horizons, rather than the direct import of federated optimization constraints. These requirements set up the broader question of how safety mechanisms behave when the threats they were designed against are only a subset of those actually present.

8.3 Composability of Robust Aggregation, Adversarial Detection, and Privacy Constraints at the Orchestration Layer

Whether the defensive mechanisms inherited from federated learning and single-agent security—robust aggregation, adversarial detection, and privacy constraints—can be composed into a coherent protection layer for collaborative orchestration remains an open question, and the evidence assembled here suggests that they cannot be combined additively without introducing new failure modes at their interfaces. Each mechanism was designed against a threat model that assumes a particular locus of adversarial control; in multi-agent orchestration, that locus shifts continuously between messages, tools, and persistent state, so a defense calibrated for one surface may be blind to, or actively undermined by, the operation of another. This section examines the three families of mechanisms in turn, identifies where their assumptions diverge, and characterizes the gaps that emerge when they are deployed together.

Robust aggregation presupposes that adversarial influence arrives as a bounded perturbation to a well-defined contribution, an assumption that communication-level attacks in multi-agent systems violate at the outset. Red-teaming work on inter-agent communication shows that an adversary can manipulate the semantic content of messages rather than their statistical distribution, so aggregation rules that trim outliers or weight contributions by similarity offer little purchase when the malicious signal is fluent and contextually plausible [99]. Earlier analysis of adversarial attacks on multi-agent communication reached a compatible conclusion from the opposite direction: the attack surface is the communication channel itself, and defenses framed around parameter updates do not transfer to natural-language coordination [55]. The contrast with gradient-based threat models is instructive, because the privacy literature supplies the sharpest illustration of how much a defense depends on its assumed channel. Gradient inversion attacks demonstrate that reconstructions are possible from shared updates alone [103], yet the cryptographic protocols developed to prevent exactly this leakage operate over arithmetic or secret-shared representations [104]. Neither machinery applies to a transcript in which agents exchange instructions and partial plans, which indicates that the aggregation layer of orchestration lacks a defensible analogue rather than merely a weaker one.

Adversarial detection, by contrast, can be adapted to the orchestration layer, but its composition with aggregation exposes a coverage gap that neither mechanism closes alone. Benchmarks for adversarial risk in multi-agent LLM systems reveal that attacks distribute themselves across turns and roles, so a detector monitoring individual messages observes only fragments of an attack whose signature is visible solely at the trajectory level [56]. Detection tuned to this level, however, tends to be expensive and retrospective, flagging behavior after the coordinated action has already been taken. A further complication arises from poisoning of the tool and context layer: realistic evaluations of model-context-protocol poisoning show that compromised external resources can inject adversarial content that appears to originate from a trusted tool, which means a detector trained on agent-to-agent traffic may treat the poisoned input as legitimate grounding. Unified security analysis of the protocol layer corroborates that the tool interface is a distinct attack surface with its own failure taxonomy [100], and covert communication protocols show that agents can establish event-driven channels that evade content-level inspection altogether [102]. Detection therefore covers the message surface while aggregation nominally covers the contribution surface, and the poisoning and covert-channel results mark the region between them as unguarded.

Privacy constraints introduce a third assumption that is frequently in tension with the other two: that the protected quantity is a fixed artifact whose exposure can be bounded once, rather than a behavior

that persists across an interaction horizon. Reviews of trust, risk, and security management for agentic systems emphasize that privacy in orchestration concerns what agents reveal through their actions over time, not only what they transmit in any single exchange [80]. This temporal framing collides with detection, which requires visibility into the very traces that privacy constraints seek to limit, and with aggregation, which needs contributions in a form comparable enough to be filtered. The tension is not merely operational: a detector strong enough to catch distributed attacks must retain cross-turn context, and that retained context is itself a privacy-relevant record. The federated-learning resolution—compute over encrypted shares so that neither detection nor aggregation sees plaintext [104]—has no established counterpart for natural-language coordination, where the semantics that detection depends on are precisely what encryption would conceal.

Beyond pairwise tensions, the three mechanisms share a structural blind spot with respect to degradation that accumulates rather than arrives. Quantification of behavioral drift in multi-agent systems over extended interactions shows that performance and safety properties decay gradually, without any single turn that a detector would flag or any single contribution that an aggregation rule would reject [1]. Constraint-drift analysis makes the same point normatively: safe behavior must be maintained continuously rather than asserted at initialization, because constraints erode as agents adapt to one another [2]. Robust aggregation, adversarial detection, and privacy constraints are all framed as per-event or per-artifact guarantees, and none of them specifies a maintenance obligation over the interaction horizon. This is the deepest compositional gap identified in the literature reviewed here: the mechanisms are not merely incompatible in their assumptions, they are jointly silent on the failure mode that the empirical work identifies as most characteristic of orchestration.

Accountability mechanisms partially compensate for this silence but do not close it, and their limitations sharpen the diagnosis. Traceability and accountability work in role-specialized pipelines demonstrates that recording role-level provenance enables post hoc attribution of harmful outcomes [8], which is valuable precisely because drift and distributed attacks escape per-event defenses. Yet attribution after the fact is not prevention, and provenance records themselves constitute the kind of persistent behavioral trace that privacy constraints are meant to restrict, reproducing the detection–privacy tension in a different register. The benchmark evidence further suggests that current evaluations reward detection accuracy on isolated attacks rather than maintenance of safety under sustained adversarial pressure [56], so accountability tooling and detection tooling are optimized against metrics that do not measure the compositional property that matters.

Taken together, the evidence indicates that robust aggregation, adversarial detection, and privacy constraints are composable only under a shared threat model that none of the three currently supplies. Aggregation assumes bounded contribution perturbations that communication attacks violate [55]; detection assumes a message-level signature that poisoning and covert channels evade [102]; privacy assumes a fixed artifact rather than a behavioral trace [80]; and all three assume event-level rather than horizon-level guarantees [1]. The gap is therefore not a missing component but a missing specification: what must be maintained, over what horizon, and against which locus of control. Formulating that specification is a precondition for composing defenses at all, and it is the question to which the analysis of maintenance guarantees turns next.

Threat surfaces therefore impose integrity, provenance, and confidentiality constraints on communication, and the communication chapter examines whether current protocols satisfy them. In the reverse direction, communication measurements reveal attack surfaces absent from the threat models, so the surface list is recalibrated by coordination findings.

9 COMMUNICATION CONSTRAINTS SHAPING CONVERSATION AND INFORMATION SHARING MECHANISMS

Having established how adversarial safety and federated constraints bound multi-agent collaboration from the protection layer, this chapter turns to the coordination layer, where natural language and communication engineering determine whether collaboration succeeds in practice. The central tension is that communication constraints—ambiguity, finite context, and bandwidth or latency limits—are either explicitly modeled as first-class design objectives or silently absorbed into generic efficiency goals, and this choice reshapes how information is shared across agents. We first trace how ambiguity and context constraints degrade coordination correctness, then examine the treatment of communication engineering constraints across collaboration architectures, and finally organize information-sharing mechanisms along the coupling of shared media and the temporal control of interaction rounds.

9.1 Ambiguity and Context Limits as Coupled Threats to Collaborative Correctness

How do ambiguity in natural-language coordination and the size and availability of the context window jointly shape the correctness of multi-agent collaboration? The literature converges on a two-sided answer: ambiguity degrades correctness by forcing agents to resolve underspecified instructions through inference rather than evidence, while context limits degrade correctness by severing the shared ground on which such inference could be validated—and the two failure modes compound, because ambiguous requests demand more contextual grounding precisely when that grounding is most likely to be truncated or lost.

The first mechanism is structural: role-specialized agents receive natural-language instructions whose interpretation is not fixed by the protocol, so correctness depends on how much interpretive latitude the coordination design leaves open. MetaGPT addresses this by encoding standardized operating procedures and structured intermediate artifacts, so that each role consumes a defined schema rather than free-form prose [14]. The contrast with loosely specified role prompts is instructive: where MetaGPT constrains the interface, unconstrained designs push ambiguity resolution onto downstream agents, who must guess at intent. Traceability work makes the consequence explicit, showing that role-specialized pipelines struggle to attribute errors when instructions are interpreted differently at different stages. This suggests that ambiguity is not merely a prompting nuisance but an accountability problem: an error that originates in interpretation cannot be localized after the fact.

A second line of evidence shows that ambiguity can be managed by decomposition rather than by schema enforcement. LONGAGENT answers questions over 128k-token documents by splitting the reading task across agents, so that each agent confronts a bounded, locally unambiguous span rather than the full document [88]. The design logic is that reducing the scope of each natural-language instruction reduces the space of plausible interpretations. However, this gain is purchased at a coordination cost: the decomposition itself must be communicated, and errors in the split propagate. MultiAgentESC faces the mirror-image problem in emotional support conversation, where the relevant context is affective and diffuse rather than document-bounded, and where misreading user intent is the dominant failure. Read together, these two systems indicate that decomposition helps when ambiguity is locational and hurts when ambiguity is interpretive.

Context availability operates as a distinct constraint with its own failure signature. AgentCoord provides a visual exploration of coordination strategies and shows that strategy choices interact with how much conversational state agents can observe, so that the same coordination pattern yields different outcomes under different context provisions. This is an important qualification to architecture-centric accounts: the coordination topology alone does not determine correctness. A survey of open problems in LLM multi-agent systems reaches a compatible conclusion, identifying context sharing and grounding as persistent bottlenecks rather than solved engineering details [7]. The implication is that context window size is not a neutral capacity parameter but a design variable that reshapes which coordination strategies are viable.

The coupling between ambiguity and context is most visible where agents must coordinate under partial observability. Who2com demonstrates that learned handshake communication can select which partner to exchange information with, effectively allocating a scarce communication budget rather than broadcasting everything [105]. The mechanism is revealing: when full context cannot be shared, agents must decide what to communicate, and that decision is itself a source of ambiguity. Heterogeneous embodied collaboration confronts the same problem with agents whose capabilities and observations differ, so that a message unambiguous to the sender may be underspecified for the receiver. In both cases, correctness depends less on the volume of shared context than on whether the shared subset is sufficient to disambiguate the task at hand.

Domain-specific systems supply convergent evidence that ambiguity resolution requires explicit shared representations. ColaCare coordinates LLM agents over electronic health record modeling, a setting where ambiguous clinical language carries direct correctness costs and where agents must reconcile heterogeneous record formats [86]. MCCD similarly decomposes complex text-to-image generation across agents, where the natural-language prompt is inherently underdetermined and must be refined into compositional subgoals before rendering [43]. The parallel is that in both domains the coordination protocol does the disambiguation work that a single monolithic model would have to do implicitly. DesignGPT offers a complementary case in design tasks, where requirements arrive ambiguous and are progressively clarified through structured multi-agent exchange. These systems suggest that the practical remedy for ambiguity is procedural: force clarification into an explicit, inspectable step.

Quantitative and architectural evidence sharpens the trade-off. NADER designs neural architectures through multi-agent collaboration, a task with a well-defined evaluation signal, and shows that coordination can be effective when the objective is unambiguous and verifiable [42]. The contrast with open-ended design

or emotional support is stark: where correctness is checkable, ambiguity is bounded by the metric; where it is not, ambiguity migrates into the coordination protocol itself. This comparison indicates that reported success rates for multi-agent collaboration are partly a function of task ambiguity, not solely of coordination quality—a confound that cross-system comparisons rarely control for.

Boundary conditions and exceptions deserve attention. One might expect that larger context windows would monotonically improve correctness by making more grounding available. The evidence is more nuanced: AgentCoord's exploration suggests that additional context can complicate strategy selection rather than simplify it [27], and the open-problems survey identifies context management as an active research gap rather than a capacity problem awaiting hardware [7]. A plausible reading is that context helps only when it is relevant and attributable; irrelevant context adds interpretive surface without adding disambiguation. This remains an inference rather than an established result, and the literature does not yet isolate context volume from context relevance in controlled comparisons.

Taken together, the evidence supports a layered account. Ambiguity is managed by constraining interfaces [14], by decomposing tasks into bounded spans, and by making clarification explicit [19]; context limits are managed by selective communication [105] and by shared representations [86]. The unresolved gap is that no study in this set manipulates ambiguity and context availability jointly, so their interaction remains inferred from separate designs rather than measured. This gap motivates the next section's focus on how information is selectively shared once context is recognized as scarce.

9.2 Latency, Bandwidth, and the Fate of Delay-Free Assumptions in Collaborative Architectures

How do communication-engineering constraints—latency, bandwidth, dedicated control channels, and the convenient fiction of delay-free message passing—survive translation into multi-agent collaboration architectures? Across the literature reviewed here, the dominant pattern is not faithful constraint modeling but systematic simplification: architectures either absorb these constraints into structural design choices or dissolve them into idealized assumptions, and the few systems that confront them directly reveal why the simplifications matter.

The most consequential simplification is the treatment of communication as instantaneous and costless. MetaGPT encodes human-standard operating procedures as structured message schemas exchanged among role-specialized agents, so that coordination proceeds through fixed-format artifacts rather than negotiation over channel availability [14]. This design choice effectively removes latency from the model: an agent's output is assumed to be immediately consumable by every downstream role. The same assumption underpins DesignGPT's division of design labor across roles [19] and the role-specialized pipelines analyzed for traceability. In each case, the absence of an explicit channel model is not neutral—it shifts the burden of coordination failure onto prompt design and role specification, leaving timing-induced failure modes invisible to the architecture.

A second, subtler simplification concerns bandwidth, which appears in these systems as context-window scarcity rather than as a transmission-rate constraint. LongAgent confronts the 128k-token limit of long-document question answering by partitioning the document among collaborating agents and aggregating their findings [88]. Here the bandwidth constraint is real and binding, but it is reframed as an information-routing problem: which agent holds which fragment, and how fragments are merged. This reframing is productive but incomplete, because it models capacity without modeling contention—no agent in this design competes with another for a shared channel, so the queuing and prioritization phenomena familiar from communication engineering have no counterpart.

Dedicated control channels find their closest architectural analogue in explicit coordination layers that are separated from task content. AgentCoord makes this separation visible by treating coordination strategy itself as a design object that practitioners explore and compare [27], while NADER instantiates a dedicated search-and-evaluation protocol through which agents propose and critique candidate architectures [42]. In both, a control plane is architecturally distinct from the data plane of task outputs. However, these control channels are logical rather than physical: they carry no latency budget, no reserved capacity, and no failure semantics, so the analogy to dedicated signaling breaks down precisely where engineering guarantees would begin.

The one line of work that models communication constraints explicitly comes from collaborative perception rather than language-agent systems. Who2com learns a handshake mechanism that decides which peers should exchange information and in what order, making communication bandwidth a first-class variable that the policy must economize [105]. This is a genuine contrast with the LLM-centric architectures: Who2com optimizes whether and with whom to communicate under a budget, whereas MetaGPT-style systems assume all-to-all availability and optimize what to say. The comparison suggests that learnable

communication gating is the mechanism most likely to transfer, yet no reviewed LLM system adopts it, leaving a clear architectural gap.

Heterogeneous embodied collaboration exposes a further dimension that delay-free assumptions obscure. When agents with different sensing and actuation capabilities cooperate, the value of a message depends on when it arrives relative to the recipient's action window. A late message is not merely delayed but semantically stale, and the architecture must decide whether to act on stale information or wait. Language-agent frameworks rarely confront this trade-off because their turns are serialized by construction; embodiment makes the temporal coupling explicit and, in doing so, shows what the serialized-turn abstraction hides.

Domain-specific deployments illustrate how these simplifications propagate into practice. ColaCare coordinates specialist agents over electronic health record data [86], MultiAgentESC structures multi-agent support for emotional conversation [87], and MCCD decomposes complex text-to-image generation across collaborating agents [43]. In each, coordination is organized around content dependencies—which agent needs which intermediate result—rather than around communication resources. The result is architectures that are robust to content complexity but blind to communication cost, an asymmetry that matters most in latency-sensitive settings such as emotional support, where a delayed response degrades the interaction itself.

Survey-level analyses corroborate that these gaps are structural rather than incidental. The challenges identified for LLM multi-agent systems include coordination overhead, communication efficiency, and the difficulty of evaluating inter-agent protocols [7]. Read against the architectural evidence above, this diagnosis implies that delay-free assumptions are not a temporary modeling convenience but a load-bearing simplification: removing it would require architectures to specify timing, capacity, and failure semantics that current designs leave implicit. The traceability literature points in the same direction, since accountability for a decision requires knowing not only which agent produced it but when and through which channel it traveled.

A cross-cutting comparison sharpens the picture. Systems with explicit control planes (AgentCoord, NADER) gain inspectability but pay in coordination overhead [27, 42]; systems with implicit, procedure-driven coordination (MetaGPT, DesignGPT) gain throughput but lose the ability to reason about timing and contention [14, 19]. Neither camp models bandwidth as a shared resource, and only the perception-oriented handshake approach treats communication selection as a learned, budgeted decision [105]. This suggests a design axis—explicit versus implicit communication control—along which current architectures cluster at the extremes and few occupy the middle.

The boundary conditions deserve emphasis. Delay-free assumptions are defensible when turns are serialized and tasks are decomposable, as in document partitioning [88] or architecture search [42], because the coordination graph is acyclic and each message has exactly one consumer. They become untenable when agents act concurrently on shared state, as in embodied settings [11], or when the interaction is itself the product, as in emotional support [87]. The literature therefore does not so much resolve the constraint question as partition it: some regimes tolerate the fiction, others quietly depend on its failure being rare.

Taken together, the evidence indicates that communication-engineering constraints are handled in three ways: absorbed into structure (procedure-driven message schemas), reframed as content routing (context partitioning), or modeled explicitly only in the perception line of work (learned handshakes). No reviewed architecture specifies latency budgets, channel contention, or message-failure semantics for language agents, and the delay-free assumption remains the default rather than a justified choice. This unresolved treatment of timing and capacity sets up the next concern: once messages are assumed free, the question of what information agents should selectively disclose—and to whom—becomes the operative design problem.

9.3 Structured Handoff and Shared Memory as Error-Attenuation Interfaces

What design dimensions characterize information-sharing mechanisms—shared memory, structured handoff, and turn limits—and how do these dimensions govern the suppression of error propagation across an agent collective? The evidence reviewed here suggests that these three mechanisms are not interchangeable conveniences but distinct interfaces, each filtering a different class of failure before it can compound downstream.

Shared memory addresses the problem of redundant re-derivation by giving agents a common, persistent substrate of intermediate artifacts. MetaGPT operationalizes this through standardized operating procedures and structured document artifacts, so that downstream roles consume specifications rather than reconstructing them from dialogue [14]. The mechanism's error-suppression value lies in reducing the number of independent inferences that could each introduce divergence: when a requirement is encoded

once as an artifact, the variance that would arise from repeated paraphrase is eliminated at the source. However, this same consolidation creates a single point of contamination—an erroneous artifact propagates to every consumer with high fidelity, which is precisely the opposite of what redundancy would provide.

Structured handoff, by contrast, constrains what crosses an agent boundary, and its design determines whether errors are localized or amplified. Traceability analyses of role-specialized pipelines show that when handoffs carry explicit provenance and accountability metadata, failures can be attributed to the producing role rather than diffusing ambiguously across the pipeline [8]. This is a qualitatively different suppression strategy from shared memory: instead of preventing divergence, it makes divergence detectable and localizable. The two mechanisms therefore trade off along a detection–prevention axis, and systems that rely on only one leave the complementary failure mode unaddressed.

Turn limits constitute the third dimension, and their effect is best understood as a cap on the depth of error compounding rather than on error occurrence. Long-document question answering frameworks that decompose work across agents must bound the number of interaction rounds, because each additional round offers another opportunity for a misinterpretation to be inherited and elaborated [88]. The limit does not improve any individual agent's reliability; it restricts the horizon over which an early mistake can be recursively reinforced. This distinguishes turn limits from both memory and handoff design, which act on content rather than on interaction depth.

A direct comparison across these dimensions reveals a coupling that single-mechanism accounts miss. Shared memory without structured handoff yields consistent but unattributable state; structured handoff without memory forces costly re-derivation at each boundary; turn limits without either merely truncate a process whose content is still unconstrained. The coordination-strategy exploration in AgentCoord makes this coupling visible by treating communication topology and message content as jointly configurable design variables, showing that coordination outcomes depend on their interaction rather than on either alone [27]. The implication is that error suppression is an interface property, not a property of any single sharing primitive.

Heterogeneous teams sharpen the case for structured handoff, because role asymmetry means that an error introduced in one modality or expertise domain may be undetectable by the receiving agent. Heterogeneous embodied collaboration demonstrates that agents with differing capabilities require explicit protocols for reconciling divergent observations before acting [11]. Where such protocols are absent, a perceptual error in one agent is not corrected but merely forwarded, and the heterogeneity that was supposed to provide complementary checking instead provides complementary blind spots.

Domain-specific deployments provide convergent evidence that handoff structure carries the suppression burden. In emotional support conversation, MultiAgentESC assigns distinct roles to intent understanding and response generation, so that a misread emotional state is caught at the role boundary rather than surfacing in the final utterance [87]. ColaCare similarly partitions electronic health record modeling across specialized agents whose outputs are cross-checked before aggregation [86]. Both cases indicate that the benefit arises from the boundary's verification function, not from the mere multiplication of agents—a distinction that purely role-counting evaluations would obscure.

Design and engineering domains show the same pattern under tighter correctness constraints. DesignGPT structures designer–researcher–engineer roles with defined artifact exchanges, and its reported gains depend on those exchanges being formalized rather than conversational [19]. NADER's architecture-search agents likewise require that candidate designs be handed off in evaluable form, since an informally described candidate cannot be validated by the receiving agent [42]. In both settings, the handoff format is what makes downstream verification possible at all.

Perception-oriented collaboration offers a useful contrast because it makes the communication channel itself the object of design. Who2com learns a handshake that decides which partner receives which message under bandwidth constraints, and its results show that selective disclosure can outperform broadcast even when broadcast is feasible [105]. This is a stronger claim than the language-agent literature typically supports: it suggests that the selectivity of a handoff, not only its structure, contributes to suppressing the propagation of low-value or misleading information. Whether the same holds for symbolic, language-mediated handoffs remains an open question.

Compositional generation provides a boundary case in which shared state is deliberately partial. MCCD coordinates agents over separate aspects of a text-to-image task, and its design implies that full state sharing would be counterproductive because it would couple errors across otherwise independent subtasks [43]. This qualifies the shared-memory argument advanced above: consolidation suppresses variance only when the consolidated content is genuinely common, and partitioning is preferable when subtask errors are uncorrelated. The design question is thus not whether to share but which information is common enough

to warrant a shared substrate.

Survey-level assessments of the field reinforce the impression that these dimensions remain underspecified as design variables. Open-problem analyses of LLM multi-agent systems identify communication overhead and error accumulation as persistent concerns while noting the absence of principled criteria for setting turn budgets or handoff granularity [7]. The mechanisms reviewed here are therefore best characterized as empirically motivated heuristics whose interactions are observed but not yet theoretically grounded—a gap that the traceability work begins to address but does not close [8].

Taken together, the evidence supports a three-dimensional account in which shared memory suppresses divergence by consolidating state, structured handoff suppresses propagation by localizing and verifying at boundaries, and turn limits suppress compounding by bounding interaction depth. These dimensions are complementary rather than substitutable, and their misalignment—memory without attribution, handoff without shared state, limits without content constraints—reproduces the very error propagation each was meant to control. This framing raises the further question of how such interfaces behave when the number of participating agents grows and the communication graph itself becomes a design object.

9.4 Constructing a Maintenance Guarantee

The protection layer's requirement that agentic systems carry maintenance guarantees across interaction horizons can now be made concrete. We define a maintenance guarantee for horizon H as a triple (φ, E, R): φ is an invariant expressed over the observable transcript—for example, that no message crosses a trust boundary without provenance, or that the task constraint remains satisfied at every handoff; E is an append-only evidence mechanism attached to each handoff, such as a structured artifact, a shared-memory record, or a temporal monitor; and R is a recovery action triggered when E shows φ violated. The guarantee holds over H if, for every turn $t \leq H$, φ is checkable from E, and any violation triggers R within a bounded number of turns.

The works reviewed here supply the components but not the triple. Structured handoff provides E in the form of inspectable intermediate artifacts [8, 14]; shared memory provides the state over which φ can be expressed; GUARDIAN's temporal monitoring provides violation detection [82]; and constraint-drift work explains why φ must be re-established rather than asserted once [2]. What is missing is the binding: no reviewed system states φ, ties it to an evidence format, and defines the recovery action R as part of the protocol. This is the concrete gap behind the observation that safety properties are not preserved by assertion alone, and it gives the communication layer a design target that is stronger than logging the messages and weaker than a full formal proof. A minimal instantiation is available. Take φ to be "every handoff carries provenance and the task constraint is re-checked", E to be the structured handoff artifact plus its provenance record, and R to be reject-and-reask at the receiving agent. The triple is constructible from the accountability pipeline of [8] and the structured artifacts of [14], and it holds for any horizon over which the handoff schema is enforced. What remains unproven is coverage: no reviewed system states the horizon H or audits R, so the guarantee is available in principle but not reported in practice.

Communication measurements therefore feed back into evaluation and architecture, closing the directional loop that the Discussion makes explicit.

10 DISCUSSION

Taken together, the five research questions converge on two cross-cutting threads. The first is architecture–task fit: the choice among centralized and decentralized topologies and between fixed and evolving role regimes is not a matter of general superiority but of matching coordination structure to task ambiguity, horizon, and verifiability. The second is evaluation fragmentation: the benchmarks that dominate current practice were designed for single-agent, single-turn, automatically scorable tasks, so they cannot register the very failures that multi-agent systems exhibit. These two threads are not independent. Because evaluation is fragmented, claims about architectural fitness rest on heterogeneous and often non-comparable evidence, and because architectures differ in their failure surfaces, no single metric can arbitrate between them. Once scale expansion and communication graphs become objects of design, architecture–task matching is no longer merely a question of topology selection; it simultaneously exposes the fragmentation of evaluation criteria, and it is at this intersection that the two threads converge and form the main axis of the discussion that follows.

The evidence on failure and propagation sharpens the architectural conclusions rather than merely supplementing them. Drift is defined as the gradual degradation of behavior, decision quality, and inter-agent consistency over long interaction sequences, with detectable drift appearing at a median of 73 turns and accelerating thereafter [1]. This temporal profile constrains when centralized orchestration earns

its cost: a central coordinator that can re-anchor intent is most valuable precisely in the long-horizon, ambiguous regimes where drift originates, whereas the coordination overhead it introduces is least justified for short, well-specified tasks. Constraint drift, understood as the loss of operational force of safety-critical constraints across a trajectory, reinforces the same point from the safety side, since constraints decay at the same delegation, communication, memory, and optimization interfaces that centralized topologies are meant to police [2]. Conversely, the finding that structured, accountable handoff protocols can halt failure cascades and raise accuracy by more than 36 points on PythonIO indicates that some of the benefit attributed to topology is in fact attributable to protocol design [8]. Architecture and protocol are therefore coupled variables, and studies that vary one while holding the other implicit risk misattributing effects.

The communication evidence supplies a mechanism that connects the failure literature to the architectural trade-offs. Ambiguity in natural language coordination forces repeated remedial specification and lets execution drift from user intent [27], while limited context windows force splitting, compression, and truncation that produce information loss and lost-in-the-middle effects [88]. When members lack access to one another's critical context shards, they hallucinate and the error propagates upward along the collaboration chain [88, 8]. This is where the architectural debate acquires its real stakes: centralized topologies can concentrate context and reduce shard unavailability, but they also concentrate the coordination bottleneck, whereas decentralized topologies distribute context availability at the price of weaker global consistency. The demonstration that a LLaMA-2 7B-driven system can handle 128k-token question answering and outperform GPT-4 by 16.42 and 1.63 points on single- and multi-hop settings shows that context constraints are mitigable through collaboration [88], but it does not show that any particular topology is the right mitigation. Communication constraints thus function as a mediating variable between architecture and outcome, and should be reported as such.

The security evidence both extends and complicates these conclusions. TAMAS formalizes the attacker with limited background knowledge—knowledge of agent roles and available tools but not of model parameters, alignment strategy, or architecture—and shows that even this limited knowledge suffices to strike the weakest link and propagate adversarial outputs through the system [56]. Prompt-layer attacks are the most effective, with impersonation reaching 82% attack success in a swarm configuration and direct prompt injection reaching 81% in Magentic-One, whereas purely persuasive agent attacks fail entirely and colluding agents achieve only 2–16% [56]. Agent-in-the-Middle pushes the threat surface from compromising an agent to intercepting and manipulating messages, achieving above 40% success in all experiments and above 70% in most without altering agent profiles or capabilities [99]. These results cut against a naive reading of the architecture literature: decentralization is often justified by robustness to single-point failure, yet message interception and prompt-layer injection exploit exactly the communication channels that decentralization multiplies. The finding that imperceptible epsilon-bounded adversarial messages can severely degrade perception when agents share intermediate representations, with white-box attacks strongest at two agents and weakening as benign agents are added [55], further shows that robustness is a function of representation sharing rather than of topology alone. Security is therefore a cross-cutting constraint that the topology–role plane does not capture, and architectural claims that ignore it are incomplete. This is not a third architectural design axis: security constrains the admissibility of every cell in the plane rather than adding a cell dimension.

Against this background, the evaluation literature explains why cross-question synthesis has been difficult to stabilize. The seven benchmarks most often used—MMLU, MATH, GSM8K, HumanEval, MBPP, Spider, and BIRD—map onto four narrow competencies: MMLU for academic knowledge; MATH and GSM8K for mathematical reasoning; HumanEval and MBPP for function-level code generation; and Spider and BIRD for structured querying. The benchmark survey explicitly marks the early benchmarks in these categories as non-agentic [4]. Coverage is uneven rather than uniformly weak: function-level code generation is near saturation, with MetaGPT+GPT-4 reaching 85.9% on HumanEval and 87.7% on MBPP [14], and mathematical reasoning shows a similar pattern in which a saturated benchmark gives way to a harder successor on which the strongest general models average only 9.8% [4]. Structured querying, by contrast, remains far from saturated, with the most advanced agentic systems reaching only 47.78% execution accuracy on BIRD, below the 59.59% of MAC-SQL [39]. This unevenness is consequential for the other questions: a literature that validates architectures on near-saturated code benchmarks will systematically under-observe drift, cascade, and constraint decay, because those phenomena require long horizons and ambiguous tasks that the benchmarks do not contain. The fragmentation of evaluation is thus not a neutral gap but a bias that shapes which architectural and safety conclusions appear supported.

Several cross-question tensions deserve explicit statement. The architectural literature offers at least six mutually incompatible classification schemes—a three-way coordination split [78], a star/ring/graph/bus pattern set [78], an equi-level/hierarchical/nested/dynamic structural set [7], a nine-dimension scheme [20],

five workflow patterns [6], and a chain/tree/graph topology set [93]—with overlapping names such that star, hierarchical, and orchestrator-workers denote substantially the same arrangement. The nine-dimension scheme itself concedes that its dimensions are not naturally independent, since limited bandwidth makes full communication infeasible [20]. This non-orthogonality is the root of the apparent contradictions between studies: two papers can report opposing conclusions about centralization while classifying the same system differently. The reconstructed two-dimensional frame of orchestration topology and role regime covers all architectures discussed in the evidence and renders these comparisons commensurable, with the predefined-versus-learnable and individual-versus-global cross-classification of communication policy corroborating the analytical value of separating the two dimensions [20]. A second tension concerns safety: the propagation literature treats communication as the vector of degradation [1, 2], while the security literature treats it as the attack surface [56, 99], and the communication literature treats it as the remedy for context scarcity [88]. These are consistent only once communication is recognized as a design variable with simultaneous costs and benefits, not as a fixed channel.

10.1 Directional Coupling Across the Five Layers

The five layers are not five parallel literatures. Read as a stack, they form a chain in which each layer constrains the layer above and calibrates the layer below. Bottom-up constraint propagation runs from diagnosis to communication: the failure modes reviewed in Section 5 rule out architectures that cannot detect or contain them; architecture choices in Section 6 determine which quantities an evaluation can attribute to collaboration; evaluation semantics in Section 7 define what counts as evidence of safety; and safety requirements in Section 8 constrain the integrity, provenance, and confidentiality properties that communication mechanisms must provide. Top-down feedback calibration runs in the opposite direction: evaluation results recalibrate architecture and safety choices, safety failures expose evaluation metrics that were never designed to detect adversarial conditions, and communication measurements—latency, bandwidth, ambiguity—feed back into both architecture and evaluation.

This directional reading explains why the layers appear decoupled in the literature. Each community optimizes a local objective and reports it in its own metric vocabulary, so an improvement at one layer can be invisible or harmful at another. A centralized architecture that improves traceability also concentrates the coordination bottleneck and creates a single point of failure; a benchmark that saturates on single-agent accuracy cannot reveal whether collaboration contributes anything; a safety mechanism validated against prompt injection says nothing about communication-channel attacks; and a communication optimization that reduces tokens may remove the redundancy that error correction depends on. The stack makes these trade-offs comparable by locating each claim at the layer where it is produced and asking which layer above or below it constrains.

Two worked examples make the top-down direction concrete. First, evaluation recalibrates architecture: SOP-centralized pipelines reach 85.9% and 87.7% on HumanEval and MBPP [14], whereas on BIRD a different agentic system reaches only 47.78% against 59.59% for a structured baseline [39]. This is not a same-pipeline transfer experiment; it shows that benchmark saturation is task-dependent, and that an architecture validated on one benchmark family cannot be assumed to transfer to another. A design rule that conditions topology on benchmark saturation follows: centralize where the task supplies structure, and decentralize where it does not. Second, communication recalibrates both evaluation and architecture: Who2com achieves segmentation accuracy comparable to centralized methods at roughly a quarter of the bandwidth [105], and CoDe remains stable under communication delay [62]; both results imply that bandwidth and delay belong in the evaluation vector and that communication protocol is an architectural choice, not an implementation detail.

Table 6 summarizes the two directions. The table is deliberately minimal: it records the constraint each layer propagates upward and the calibration signal it receives from above. Its purpose is not to assert that every edge has been quantified—most have not—but to make the missing edges visible. The weakest edges in the current evidence are the upward path from evaluation to safety, where metric semantics are rarely adversarial, and the downward path from communication measurements to architecture, where latency and bandwidth are still treated as implementation details rather than design variables.

The reusable contribution of this review is threefold. First, it supplies a classification framework that replaces six overlapping schemes with two principal dimensions, orchestration topology and role regime, under which every architecture in the evidence can be placed and compared [78, 7, 20, 6, 93]. Second, it supplies an evaluation coordinate that plots existing benchmarks by competency coverage and agentic status, making visible both the saturated regions and the structural blind spots into which multi-agent failures fall [4]. Third, it identifies a design dimension that the architecture literature omits: the

Table 6: Directional coupling across the five layers. Each layer propagates a constraint upward and receives a calibration signal from above; the weakest edges in the current evidence are evaluation-to-safety and communication-to-architecture.

Layer	Constraint propagated upward	Calibration received from above
Failure diagnosis	Named modes rule out architectures that cannot detect or contain them (drift demands long-horizon monitoring; constraint drift demands re-anchoring)	Error-reduction evidence redefines which modes count as suppressed
Architecture	Topology and role choices determine what evaluation can attribute to collaboration	Evaluation results recalibrate topology and role choices; safety requirements constrain admissible designs
Evaluation	Metric semantics define what counts as evidence for safety and communication claims	Safety failures expose missing adversarial metrics; communication measurements add latency and bandwidth dimensions
Safety	Threat surfaces impose integrity, provenance, and confidentiality constraints on communication	Communication findings reveal attack surfaces absent from the threat models
Communication	Measured ambiguity, latency, and bandwidth outcomes feed back into evaluation metrics and architecture choices	Receives constraints from safety and calibration targets from evaluation

communication channel as a jointly security-relevant and context-relevant design surface, whose protocol properties—accountability, handoff structure, and message integrity—mediate between topology and outcome [56, 99, 8]. Together these contributions convert a set of fragmented findings into a comparable design space, and they indicate where the field's evidence is thinnest: long-horizon, ambiguous, safety-critical tasks on which architecture, communication, and security effects can be observed simultaneously rather than in isolation.

11 FUTURE RESEARCH

Future Research Directions

After integrating fragmented findings into a comparable design space and identifying where the evidence is weak, the next step is to translate these gaps into an actionable research agenda, specifying the priorities among the directions and the validation paths.

A balanced reading of the current evidence reveals that each gap is also an opening. Error-propagation control has been organised around five open problems—structured verification, structured handoff and accountability, adversarial debate, metacognitive reflection, and behavioural anchoring—yet the first three remain anchored in the verification and coordination stages of the workflow, leaving downstream execution and long-horizon drift largely unaddressed. These five problems are a research agenda, not a taxonomy of the six mechanisms reviewed in the quantitative validation section; a single mechanism can bear on more than one problem. The reported gains from reflection mechanisms are encouraging but narrow: NADER's Reflector raises network executability from 0.54 to 0.64 and quality from 0.78 to 0.84 through learning from immediate feedback [42]. Comparable structural remedies appear in role-based collaboration and meta-programming frameworks that impose explicit intermediate artefacts, while drift studies show that behavioural degradation accumulates over extended interactions. The opportunity is therefore to extend verification from workflow entry points to continuous, execution-level monitoring, and to test whether reflection gains transfer beyond the single-domain settings in which they were demonstrated.

A critical stance is warranted on how strongly the orchestration evidence supports general claims. Centralised SOP orchestration improves accuracy markedly on structured tasks—Flow-of-Action lifts root-cause-analysis accuracy from 35.50% to 64.01% with GPT-4-Turbo—and ablation is decisive, with average accuracy collapsing from 54.06% to 15.39% when SOP knowledge is removed. These results establish that fixed procedures matter, but they come from a narrow task family and a single model generation; they do not yet justify claims that centralised orchestration dominates in open-ended or adversarial settings. Decentralised and self-organising alternatives report competitive coordination without a central controller, and communication-centric surveys document how much performance depends on protocol design rather than topology alone. Likewise, the measurement base is thinner than it appears: a single "success rate" label is invoked across many measurement relationships while denoting at least three mutually exclusive semantics—exact match, operational executability, and physical collision-free completion

—and "executability" itself spans binary proportions, 1–4 ordinal scales, and compilation-pass percentages. Cross-paper comparisons of success are therefore not yet defensible, and a shared measurement model is a precondition for cumulative evidence rather than an optional refinement.

The synthesised view identifies privacy and communication as the cross-cutting levers. Shared gradients leak ground-truth labels deterministically: iDLG exploits the sign structure of cross-entropy gradients to extract true labels with 100% accuracy and outperforms earlier leakage attacks across MNIST, CIFAR-100, and LFW [103]. Client-aided two-party protocols rest entirely on the assumption that no computing party colludes with the client distributing Beaver triples, and that assumption, once violated, collapses the privacy guarantee wholesale [104]. On the communication side, Who2com treats bandwidth and CoDe treats delay as first-class modelling targets: Who2com achieves segmentation accuracy comparable to centralised methods at roughly a quarter of the bandwidth [105], while CoDe aligns intent and timeliness under communication delay [62]. The remaining studies model neither delay nor bandwidth explicitly. These findings converge on a single leverage point: security and bandwidth budgets must be co-designed with the collaboration protocol, not appended afterwards. Federated and decentralised analytics offer a natural testbed for this co-design, and agentic chaos-engineering methods provide a route to stress such guarantees under failure.

Several further directions follow from the same synthesis. Failure analysis of autonomous agents shows that breakdowns cluster around specification and coordination rather than raw model capability, which argues for benchmarks that report failure modes alongside success. Security-oriented surveys and red-teaming studies indicate that communication channels are themselves an attack surface, so protocol-level defenses should be evaluated jointly with privacy leakage. Finally, measurement harmonisation—explicit construct definitions, reported scales, and shared reporting templates—would let orchestration, reflection, and communication results be compared rather than merely juxtaposed. Pursued together, these directions convert the identified gaps into a coherent research agenda.

12 LIMITATIONS

The evidence base combines an automated retrieval pool with manual supplementation, and this combination bounds what the review can claim to cover. The automated searches drew on a fixed set of queries and index sources, so literature using different terminology, residing in venues not indexed by those sources, or published in languages other than English is likely underrepresented; manual supplementation partially offsets this by adding records identified through citation chasing and domain expertise, but the addition is opportunistic rather than systematic, and its coverage varies across subthemes. Retrieval closed at a predetermined cutoff date, and work appearing after that date is excluded except where it was already in hand. Consequently, the corpus should be read as a structured but incomplete sample of the relevant literature rather than an exhaustive census, and any apparent gaps in coverage may reflect retrieval boundaries as much as gaps in the field.

The review uses model-assisted extraction and drafting, and we have made deliberate efforts to keep that assistance auditable. Each substantive claim is annotated with the specific supporting references on which it rests, so readers can trace assertions back to source documents, and the analytical process is documented through retained intermediate records, including extraction outputs and the sequence of analytical decisions, which allow the derivation of findings to be inspected. These measures make the reasoning behind each claim checkable, but they do not eliminate error: individual extraction inaccuracies may persist, particularly where source statements are ambiguous, where terminology varies across studies, or where quantitative details are reported inconsistently. Model assistance can also introduce subtle distortions in summarization that survive spot-checking. We therefore present the synthesis as verifiable in principle and checked in practice, while acknowledging a residual and non-zero error rate in the extracted evidence.

The scope of the review is deliberately narrow. It addresses the defined topic directly and admits adjacent literatures only where they intersect with that topic, for example where a neighboring field supplies a method, a construct, or a comparison case that the core question requires. Work that is relevant to neighboring areas but does not bear on the review's central question is excluded even when it is closely related, and findings from adjacent fields are reported only to the extent that they inform the topic under review. This boundary keeps the synthesis focused, but it means the review does not survey those adjacent areas in their own right, and readers seeking comprehensive treatment of neighboring literatures should consult reviews dedicated to them.

13 CONCLUSION

Failures in large language model-driven multi-agent collaboration rarely stem from an oversight in a single component. The literature names 30 distinct modes, which we group into six families; behavioral drift, constraint drift, and cascades form the main trunk of behavioral degradation. Suppression and anchoring mechanisms have been compared quantitatively, but the strength of evidence is stratified and the evaluation conditions are mutually incomparable, making cross-cutting adjudication almost impossible. This evidence landscape suggests that failure is a product of five coupled layers—failure diagnosis and suppression, collaboration architecture and role design, evaluation benchmark and metric measurement, adversarial safety and federated constraint protection, and natural language and communication coordination—rather than an engineering defect that can be patched point by point.

At the diagnostic layer, the named failure modes are not isolated defects: drift, constraint drift, and cascades share interface-level vulnerabilities, while planning and governance failures explain why some systems never reach the interface at all. At the architectural layer, collaborations fall into a 2×2 design space along two principal dimensions; centralized and decentralized routes form a conditional Pareto trade-off among accuracy, traceability, and adaptability, with no single winner, and fixed versus evolving roles reflect different value weights on the same frontier. At the evaluation layer, coverage is narrow but deep, with single-point saturation and lateral fragmentation, and metric semantics exhibit systematic same-name-different-meaning and same-meaning-different-name phenomena; a unified framework must be decomposed into four separate dimensions—task type, collaboration scale, error tolerance, and cost—to accommodate existing benchmarks. At the safety layer, adversarial defenses and federated privacy constraints are partially composable at the orchestration level, yet structural gaps remain. At the communication layer, ambiguity and context constraints impair correctness through identifiable mechanisms, while communication engineering constraints are mostly left implicit, with only a few works modeling them as first-class design objectives—this silence itself is an evidence gap. Across the five layers, constraint propagation runs upward—failure modes constrain admissible architectures, architectures constrain what can be measured, evaluation defines what counts as safety evidence, and safety constrains communication channels—while evaluation evidence and communication findings calibrate architecture choices downward.

The core insight that emerges is: the failure of multi-agent collaboration is not an engineering defect that can be fixed at a single point, but rather a systemic property arising from the coupling among diagnostic, architectural, evaluation, safety, and communication constraints; any claim of an "optimal solution" holds only under explicitly specified trade-off coordinates and evaluation conditions. For researchers, when reporting collaboration mechanisms, they should simultaneously specify the architectural trade-off position and the evaluation-condition stratum in which the mechanism operates, so as to avoid erroneous cross-benchmark strength comparisons; for engineers, communication constraints and adversarial protection should be explicitly modeled as first-class design objectives before deployment, rather than left to post hoc patches.

A EVIDENCE AND PROVENANCE

This appendix records the source and derivation of the quantitative claims that carry the review's central argument. Each row names the claim, the primary source, and the locator or formula used to reconstruct it. Claims whose intermediate data could not be preserved were removed during revision rather than reported without provenance.

Table 7: Provenance of the review's headline quantitative claims.

Claim	Primary source	Locator / derivation
Agent drift: ASI below 0.85 at median 73 rounds; decline 0.08 then 0.19 per 50 rounds; task success −42%; interventions 3.2×; conflict +487.5%	[1]	Agent Drift full text, simulation results and per-domain tables
Domain susceptibility: 53.2%, 39.7%, 31.8% within 500 rounds	[1]	same, per-domain results
HumanEval/MBPP: 85.9%/87.7%	[14]	MetaGPT results table
BIRD: 47.78% vs 59.59% (MAC-SQL)	[39]	NL2SQL evaluation table
Validation efficiency: 51.05%	[39]	same, efficiency table
NADER: executability 0.54→0.64; quality 0.78→0.84	[42]	NADER full text, results table
Corpus: 189 screened, 105 cited	PAPER_INDEX.csv; references.bib	counts at the retrieval cutoff
Six suppression mechanisms: 2 strong / 4 weaker	Table 2	per-paper full text (repetition, variance, scale)
Failure-mode inventory: 30 named modes in six families	Table 1	aggregation of reviewed works; the 12 headline labels are the most frequent subset

The cross-task metric comparison in the validation section is qualitative: pairwise Jaccard similarity between the six metric sets is near zero, but the exact matrix is not reproduced because the sets are defined at different levels of granularity. We therefore do not report a numeric mean.

REFERENCES

[1] Abhishek Rath. Agent drift: Quantifying behavioral degradation in multi-agent llm systems over extended interactions. arXiv (Cornell University), 2026.

[2] Tianxiao Li, Yixing Ma, Haiquan Wen, Zhenglin Huang, Qianyu Zhou, Zeyu Fu, and Guangliang Cheng. Safe multi-agent behavior must be maintained, not merely asserted: Constraint drift in llm-based multi-agent systems. arXiv (Cornell University), 2026.

[3] Ruofan Lu, Yichen Li, and Yintong Huo. Exploring autonomous agents: A closer look at why they fail when completing tasks. 2025 40th IEEE/ACM International Conference on Automated Software Engineering (ASE), 2025.

[4] Mohamed Amine Ferrag, Norbert Tihanyi, and Mérouane Debbah. From llm reasoning to autonomous ai agents: A comprehensive review. IEEE Access, 2026.

[5] Felix Härer. Specification and evaluation of multi-agent llm systems - prototype and cybersecurity applications. 2025 International Conference on Cybersecurity and AI-Based Systems (Cyber-AI), 2025.

[6] Chaojia Yu, Zihan Cheng, Hanwen Cui, Yishuo Gao, Zexu Luo, Yijin Wang, Hangbin Zheng, and Yong Zhao. A survey on agent workflow – status and future. 2025 8th International Conference on Artificial Intelligence and Big Data (ICAIBD), 2025.

[7] Shanshan Han, Qifan Zhang, Weizhao Jin, and Zhaozhuo Xu. Llm multi-agent systems: Challenges and open problems. arXiv (Cornell University), 2024.

[8] Amine Barrak. Traceability and accountability in role-specialized multi-agent llm pipelines. 2025 40th IEEE/ACM International Conference on Automated Software Engineering Workshops (ASEW), 2025.

[9] XiaoJie Zhang, JianHan Wu, Xiaoyang Qu, and Jianzong Wang. Mita: A hierarchical multi-agent collaboration framework with memory-integrated task allocation. ICASSP 2026 - 2026 IEEE International Conference on Acoustics, Speech and Signal Processing (ICASSP), 2026.

[10] Zhao Mandi, Shreeya Jain, and Shuran Song. Roco: Dialectic multi-robot collaboration with large language models. 2024 IEEE International Conference on Robotics and Automation (ICRA), 2024.

[11] Xinzhu Liu, Di Guo, Xinyu Zhang, and Huaping Liu. Heterogeneous embodied multi-agent collaboration. IEEE Robotics and Automation Letters, 2024.

[12] Di Wu, Xian Wei, Guang Chen, Hao Shen, X.-S Wang, Wenhao Li, and Bo Jin. Generative multi-agent collaboration in embodied ai: A systematic review. arXiv (Cornell University), 2025.

[13] Eser Kandogan, Nikita Bhutani, Dan Zhang, Rafael Li Chen, Sairam Gurajada, and Estevam Hruschka. Orchestrating agents and data for enterprise: A blueprint architecture for compound ai. 2025 IEEE 41st International Conference on Data Engineering Workshops (ICDEW), 2025.

[14] Sirui Hong, Mingchen Zhuge, Jiaqi Chen, Xiawu Zheng, Yuheng Cheng, Ceyao Zhang, Jinlin Wang, Zili Wang, Steven Ka Shing Yau, Zijuan Lin, Liyang Zhou, and Chenyu Ran. Metagpt: Meta programming for a multi-agent collaborative framework. arXiv (Cornell University), 2023.

[15] Xianghua Zeng, Hao Ming Peng, and Angsheng Li. Effective and stable role-based multi-agent collaboration by structural information principles. Proceedings of the AAAI Conference on Artificial Intelligence, 2023.

[16] Jiaxi Cui, Munan Ning, Zongjian Li, Hao Li, Yan Yang, Bohua Chen, Bin Ling, Yonghong Tian, and Li Yuan. Chatlaw: A multi-agent legal assistant based on a role-aligned mixture-of-experts architecture. Fundamental Research, 2026.

[17] Thanh-Long Bui, Hoa Khanh Dam, and Rashina Hoda. An llm-based multi-agent framework for agile effort estimation thanh-long bui. 2025 40th IEEE/ACM International Conference on Automated Software Engineering (ASE), 2025.

[18] Jeshwanth Challagundla, Mantek Singh, Siddharth Raina, Smarth Behl, FNU Harsh, and Jasmin Jarsania. Si-agent: An agentic framework for feedback-driven generation and tuning of human-readable system instructions for large language models. 2025 16th International Conference on Information, Intelligence, Systems & Applications (IISA), 2025.

[19] Shiying Ding, Xinyi Chen, Yan N. Fang, Wenrui Liu, Yiwu Qiu, and Chunlei Chai. Designgpt: Multi-agent collaboration in design. 2023. Preprint.

[20] Changxi Zhu, Mehdi M. Dastani, and Shihan Wang. A survey of multi-agent deep reinforcement learning with communication. Autonomous Agents and Multi-Agent Systems, 2024.

[21] Emanuele Pesce and Giovanni Montana. Improving coordination in small-scale multi-agent deep reinforcement learning through memory-driven communication. Machine Learning, 2020.

[22] Yiming Li, Shunli Ren, Pengxiang Wu, Siheng Chen, Feng Chen, and Wenjun Zhang. Learning distilled collaboration graph for multi-agent perception yiming li. arXiv (Cornell University), 2021.

[23] Marwa Chafii, Salmane Naoumi, Réda Alami, Ebtesam Almazrouei, Mehdi Bennis, and Mérouane Debbah. Emergent communication in multi-agent reinforcement learning for future wireless networks. IEEE Internet of Things Magazine, 2023.

[24] Zongxi Liu and Hongyang Du. Model context protocol-based internet of experts for wireless environment-aware llm agents. GLOBECOM 2025 - 2025 IEEE Global Communications Conference, 2025.

[25] Luis Miguel Vieira Da Silva, Aljosha Köcher, and Felix Gehlhoff. Beyond formal semantics for capabilities and skills: Model context protocol in manufacturing. 2025 IEEE 30th International Conference on Emerging Technologies and Factory Automation (ETFA), 2025.

[26] Jie Zhu, Yimin Tian, Boyang Li, Kehao Wu, Zhongzhi Liang, Junhui Li, Xianyin Zhang, Lifan Guo, Feng Chen, Yong Liu, and Chi Zhang. Finmcp-bench: Benchmarking llm agents for real-world financial tool use under the model context protocol. ICASSP 2026 - 2026 IEEE International Conference on Acoustics, Speech and Signal Processing (ICASSP), 2026.

[27] Bo Pan, Jiaying Lu, Ke Wang, Zheng Li, Zhen Wen, Yingchaojie Feng, Minfeng Zhu, and Wei Chen. Agentcoord: Visually exploring coordination strategy for llm-based multi-agent collaboration. arXiv (Cornell University), 2024.

[28] Yiming Li, Dekun Ma, Ziyan An, Zixun Wang, Yiqi Zhong, Siheng Chen, and Chen Feng. V2x-sim: Multi-agent collaborative perception dataset and benchmark for autonomous driving. IEEE Robotics and Automation Letters, 2022.

[29] Ahmed Allam, Youssef Mansour, and Mohamed Shalan. Asic-agent: An autonomous multi-agent system for asic design with benchmark evaluation. 2025 IEEE International Conference on LLM-Aided Design (ICLAD), 2025.

[30] Yujie Zhao, Hejia Zhang, Hanxian Huang, Zhongming Yu, and Jishen Zhao. Mage: A multi-agent engine for automated rtl code generation. 2025 62nd ACM/IEEE Design Automation Conference (DAC), 2025.

[31] Jinwei Tang, Jiayin Qin, Nuo Xu, Pragnya Sudershan Nalla, Yu Cao, Yang Katie Zhao, and Caiwen Ding. Mahl: Multi-agent llm-guided hierarchical chiplet design with adaptive debugging. 2025 IEEE/ACM International Conference On Computer Aided Design (ICCAD), 2025.

[32] Liutong Han, Zhiyuan Tan, Hongbin Zhang, Pengcheng Wang, Chu Kang, Mingjie Xing, and Yanjun Wu. Intrintrans: Llm-based intrinsic code translator for risc-v vector. 2026 29th International Conference on Computer Supported Cooperative Work in Design (CSCWD), 2026.

[33] Myeongsoo Kim, Tyler Stennett, Saurabh Sinha, and Alessandro Orso. A multi-agent approach for rest api testing with semantic graphs and llm-driven inputs. 2025 IEEE/ACM 47th International Conference on Software Engineering (ICSE), 2025.

[34] Rabimba Karanjai, Lei Xu, and Weidong Shi. Hpcagenttester: A multi-agent llm approach for enhanced hpc unit test generation. 2025 2nd IEEE/ACM International Conference on AI-powered Software (AIware), 2025.

[35] Takaaki Toda and Tatsuya Mori. Chase: Llm agents for dissecting malicious pypi packages. 2025 2nd IEEE/ACM International Conference on AI-powered Software (AIware), 2025.

[36] Youssef Maklad, Fares Wael, Ali Hamdi, Wael Elsersy, and Khaled Shaban. Multifuzz: A dense retrieval-based multi-agent system for network protocol fuzzing. 2025 IEEE/ACS 22nd International Conference on Computer Systems and Applications (AICCSA), 2025.

[37] Eduardo Almeida Palmieri, Mohamed Chahine Ghanem, Viktor Sowinski-Mydlarz, and Dipo Dunsin. A framework for embedding generative and agentic ai in open source intelligence. 2025 7th International Conference on Blockchain Computing and Applications (BCCA), 2025.

[38] Sebastián R. Castro, Roberto Campbell, Nancy Lau, Octavio Villalobos, Jiaqi Duan, and Alvaro A. Cardenas. Large language models are autonomous cyber defenders. 2025 IEEE Conference on Artificial Intelligence (CAI), 2025.

[39] David Onyango and Naseef Mansoor. An agentic system for schema aware nl2sql generation. 2026 IEEE Conference on Artificial Intelligence (CAI), 2026.

[40] Mohammad Nour Al Awad, Sergey Ivanov, Olga Tikhonova, and Ivan Khodnenko. A multimodal conversational agent for tabular data analysis. 2025 IEEE International Conference on Data Mining Workshops (ICDMW), 2025.

[41] Kun Ouyang, Haoyu Wang, and Dong Fang. Fela: A multi-agent evolutionary system for feature engineering of industrial event log data. 2026 IEEE 42nd International Conference on Data Engineering (ICDE), 2026.

[42] Zekang Yang, Wang Zeng, Sheng Jin, Chen Qian, Ping Luo, and Wentao Liu. Nader: Neural architecture design via multi-agent collaboration. 2025 IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR), 2025.

[43] M. H. Li, Xiaolu Hou, Ziyang Liu, Dingkang Yang, Ziyun Qian, Jiawei Chen, Jinjie Wei, Yue Jiang, Qingyao Xu, and Lihua Zhang. Mccd: Multi-agent collaboration-based compositional diffusion for complex text-to-image generation. 2025. Preprint.

[44] Yiqun Zhang, Xiaocui Yang, Shi Feng, Daling Wang, Yifei Zhang, and Kaisong Song. Can llms beat humans in debating? a dynamic multi-agent framework for competitive debate. ICASSP 2026 - 2026 IEEE International Conference on Acoustics, Speech and Signal Processing (ICASSP), 2026.

[45] Amal Feriani and Ekram Hossain. Single and multi-agent deep reinforcement learning for ai-enabled wireless networks: A tutorial. IEEE Communications Surveys & Tutorials, 2021.

[46] Mateus P. Mota, Álvaro Valcarce, Jean-Marie Gorce, and Jakob Hoydis. The emergence of wireless mac protocols with multi-agent reinforcement learning. 2021 IEEE Globecom Workshops (GC Wkshps), 2021.

[47] Luciano Miuccio, Salvatore Riolo, Sumudu Samarakoon, Daniela Panno, and Mehdi Bennis. Learning generalized wireless mac communication protocols via abstraction. GLOBECOM 2022 - 2022 IEEE Global Communications Conference, 2022.

[48] Farhad Rezazadeh, Amir Ashtari Gargari, Sandra Lagén, Houbing Song, Dusit Niyato, and Lingjia Liu. Toward generative 6g simulation: An experimental multi-agent llm and ns-3 integration. 2025 IEEE International Mediterranean Conference on Communications and Networking (MeditCom), 2025.

[49] Masoud Shokrnezhad and Tarik Taleb. An autonomous network orchestration framework integrating large language models with continual reinforcement learning. IEEE Communications Magazine, 2025.

[50] Jingzhi Hu and Geoffrey Ye Li. Distillation-enabled knowledge alignment protocol for semantic communication in ai agent networks. IEEE Communications Letters, 2025.

[51] Haichao Ji, Zibo Wang, Cheng Pan, Meng Han, Yifei Zhu, Dan Wang, and Zhu Han. Lafa: Agentic llm-driven federated analytics over decentralized data sources. 2025 IEEE International Conference on Cloud Computing Technology and Science (CloudCom), 2025.

[52] Joshua Owotogbe. Assessing and enhancing the robustness of llm-based multi-agent systems through chaos engineering. 2025. Preprint.

[53] Dong Qiu, Duo Xu, and Limengxi Yue. Reinforcement learning-augmented llm agents for collaborative decision making and performance optimization. 2025 7th International Conference on Frontier Technologies of Information and Computer (ICFTIC), 2025.

[54] Renan Souza, Amal Gueroudji, Stephen DeWitt, Daniel Rosendo, Tirthankar Ghosal, Robert Ross, Prasanna Balaprakash, and Rafael Ferreira Da Silva. Prov-agent: Unified provenance for tracking ai agent interactions in agentic workflows. 2025 IEEE International Conference on eScience (eScience), 2025.

[55] James Z. Tu, Tsun-Hsuan Wang, Jingkang Wang, Sivabalan Manivasagam, Mengye Ren, and Raquel Urtasun. Adversarial attacks on multi-agent communication. 2021 IEEE/CVF International Conference on Computer Vision (ICCV), 2021.

[56] Ishan Kavathekar, Hemang Jain, Ameya Rathod, Ponnurangam Kumaraguru, and Tanuja Ganu. Tamas: Benchmarking adversarial risks in multi-agent llm systems. 2026. Preprint.

[57] Junlin Wang, Zehao Wu, Shaowei Lu, Yanlan Li, and Xinghao Huang. Sirag: Towards stable and interpretable rag with a process-supervised multi-agent framework. 2025 International Conference on Intelligent Robotics and Automatic Control (IRAC), 2025.

[58] Shi-Qi Yan, Jia-Chen Gu, Yun Zhu, and Zhen-Hua Ling. Corrective retrieval augmented generation. Preprint.

[59] Philseok Lee, Mina Son, and Zihao Jia. Ai-powered automatic item generation for psychological tests: A conceptual framework for an llm-based multi-agent aig system. Journal of Business and Psychology, 2025.

[60] Hanjiang Hu, Changliu Liu, Na Li, and Yebin Wang. Training task reasoning llm agents for multi-turn task planning via single-turn reinforcement learning. IEEE Control Systems Letters, 2025.

[61] Yuan Zhuang, Yi Shen, Zhili Zhang, Yuxiao Chen, and Fei Miao. Yolo-marl: You only llm once for multi-agent reinforcement learning. 2025 IEEE/RSJ International Conference on Intelligent Robots and Systems (IROS), 2025.

[62] Shuai Song, Youfang Lin, Sheng Han, Chang Yao, Hao Wu, Shuo Wang, and Kai Lv. Code: Communication delay-tolerant multi-agent collaboration via dual alignment of intent and timeliness. Proceedings of the AAAI Conference on Artificial Intelligence, 2025.

[63] Dae Cheol Kwon and Xinyu Zhang. Cp-agentnet: Autonomous and explainable communication protocol design using generative agents. 2025. Preprint.

[64] Yijun Liu, Wu Liu, Xiaoyan Gu, Hantao Yao, Jiebo Luo, and Yongdong Zhang. Rumorsphere: A framework for million-scale agent-based dynamic simulation of rumor propagation. IEEE Transactions on Multimedia, 2026.

[65] Ziqi Liu, Ziyang Zhou, Yilin Li, Haiyang Zhang, and Yangbin Chen. Following the trace: A structured path to empathetic response. Preprint.

[66] Yefeng Wu, Yuchen Song, Yecheng Zhao, Ling Wu, and Shan Wan. Cogevo-edu: Cognitive evolution educational multi-agent collaborative system. 2026 29th International Conference on Computer Supported Cooperative Work in Design (CSCWD), 2026.

[67] Jipeng Cen, Jiaxin Liu, Zhixu Li, and Jingjing Wang. Sqlfixagent: Towards semantic-accurate text-to-sql parsing via consistency-enhanced multi-agent collaboration jipeng cen. Proceedings of the AAAI Conference on Artificial Intelligence, 2025.

[68] Chia-Tung Ho, Jing Gong, Yunsheng Bai, Chenhui Deng, Haoxing Ren, and Brucek Khailany. Marco: Configurable graph-based task solving and multi-ai agents framework for hardware design. 2025 Symposium on VLSI Technology and Circuits (VLSI Technology and Circuits), 2025.

[69] Toqeer Ali Syed, Mohammad Riyaz Belgaum, Salman Jan, Asadullah Abdullah Khan, and Saad Said Alqahtani. Agentic ai for autonomous defense in software supply chain security: Beyond provenance to vulnerability mitigation 1st t. 2025 International Conference on Computer and Applications (ICCA), 2025.

[70] Sizhe Liu, Yizhou Lu, Siyu Chen, Xiyang Hu, Jieyu Zhao, Tianfan Fu, and Yue Zhao. Drugagent: Automating ai-aided drug discovery programming through llm multi-agent collaboration. arXiv (Cornell University), 2024.

[71] Longtao Zhu, Hongyu Yang, Ge Song, Xin Ma, Yanxin Zhang, Yulong Ji, and Jinchang Ren. Autofpdesigner: Automated flight procedure design based on multi-agent large language model. IEEE Transactions on Intelligent Transportation Systems, 2026.

[72] Minghao Wu, Jiahao Xu, Yulin Yuan, Gholamreza Haffari, Longyue Wang, Weihua Luo, and Kaifu Zhang. (perhaps) beyond human translation: Harnessing multi-agent collaboration for translating ultra-long literary texts mingh. arXiv (Cornell University), 2024.

[73] Peng Wang, Shuo Li, Ruoxi Zhou, Qiguang Chen, Xu Xiao , Hao Fei, Dagang Li, Wanxiang Che, and Libo Qin. Improving consistency identification in task-oriented dialogue through multi-agent collaboration. 2025. Preprint.

[74] Mariusz Kaleta. Toward conversational decision support systems: Integrating llms in the operations research methodology mariusz kaleta. 2025 20th Conference on Computer Science and Intelligence Systems (FedCSIS), 2025.

[75] Sen Lin, Mehmet Dedeoğlu, and Junshan Zhang. Accelerating distributed online meta-learning via multi-agent collaboration under limited communication sen lin. 2021. Preprint.

[76] N Crawford, Edward Duffy, Iman Evazzade, Torsten Foehr, Gregory K. Robbins, Debbrata K. Saha, J. Shashank Varma, and Marcin Ziółkowski. Bmw agents - a framework for task automation through multi-agent collaboration noel crawford. arXiv (Cornell University), 2024.

[77] Raphael Shu, Nilaksh Das, Michelle Yuan, Monica Sunkara, and Yi Zhang. Towards effective genai multi-agent collaboration: Design and evaluation for enterprise applications. arXiv (Cornell University), 2024.

[78] Yingxuan Yang, Qiuying Peng, Jun Wang, Ying Wen, and Weinan Zhang. Llm-based multi-agent systems: Techniques and business perspectives. arXiv (Cornell University), 2024.

[79] Mert Cemri, Melissa Z. Pan, Shuyi Yang, Lakshya A. Agrawal, Bhavya Chopra, Rishabh Tiwari, Kurt Keutzer, Aditya Parameswaran, Dan Klein, Kannan Ramchandran, Matei Zaharia, Joseph E. Gonzalez, and Ion Stoica. Why do multi-agent llm systems fail? arXiv (Cornell University), 2025.

[80] Shaina Raza, Ranjan Sapkota, Manoj Karkee, and Christos Emmanouilidis. Trism for agentic ai: A review of trust, risk, and security management in llm-based agentic multi-agent systems. arXiv (Cornell University), 2025.

[81] Bingyu Yan, Zhibo Zhou, Litian Zhang, Lian Zhang, Ziyi Zhou, Dezhuang Miao, Zhoujun Li, Chaozhuo Li, and Xiaoming Zhang. Beyond self-talk: A communication-centric survey of llm-based multi-agent systems. arXiv (Cornell University), 2025.

[82] Jialong Zhou, Lichao Wang, and Xiao Yang. Guardian: Safeguarding llm multi-agent collaborations with temporal graph learning. 2025. Preprint.

[83] Minfeng Qi, Tianqing Zhu, Lefeng Zhang, Ningran Li, Yu an Tan, and Wanlei Zhou. Towards transparent and incentive-compatible collaboration in decentralized llm multi-agent systems: A blockchain-driven approach. IEEE Transactions on Network Science and Engineering, 2026.

[84] Daisuke Kikuta, Hiroki Ikeuchi, and Kengo Tajiri. Llm-powered fully automated chaos engineering: Towards enabling anyone to build resilient software systems at low cost. 2025 40th IEEE/ACM International Conference on Automated Software Engineering (ASE), 2025.

[85] Xudong Guo, Kaixuan Huang, Jiale Liu, Wenhui Fan, Natalia Vélez, Qingyun Wu, Huazheng Wang, Thomas L. Griffiths, and Mengdi Wang. Embodied llm agents learn to cooperate in organized teams. IEEE Transactions on Computational Social Systems, 2026.

[86] Zixiang Wang, Yinghao Zhu, Hang Zhao, Xiaochen Zheng, Dehao Sui, Tianlong Wang, Wen Tang, Yasha Wang, Ewen Munro Harrison, Chengwei Pan, Junyi Gao, and Liantao Ma. Colacare: Enhancing electronic health record modeling through large language model-driven multi-agent collaboration. 2025. Preprint.

[87] Yangyang Xu, Jinpeng Hu, Zhuoer Zhao, Zhangling Duan, Xiao Sun, and Xun Yang. Multiagentesc: A llm-based multi-agent collaboration framework for emotional support conversation. 2025. Preprint.

[88] Jun Zhao, Can Zu, Xu Hao, Yi Lu, Wei He, Yiwen Ding, Tao Gui, Qi Zhang, and Xuanjing Huang. Longagent: Achieving question answering for 128k-token-long documents through multi-agent collaboration. 2024. Preprint.

[89] Yingxuan Yang, Huacan Chai, Shuai Shao, Yuanyi Song, Siyuan Qi, Renting Rui, and Weinan Zhang. Agentnet: Decentralized evolutionary coordination for llm-based multi-agent systems. 2025. Preprint.

[90] Zhexuan Wang, Yutong Wang, Xuebo Liu, Liang Ding, Miao Zhang, Jie Liu, and Min Zhang. Agentdropout: Dynamic agent elimination for token-efficient and high-performance llm-based multi-agent collaboration. 2025. Preprint.

[91] Yufan Dang, Qian Chen, X. L. Luo, Jingru Fan, Zihao Xie, Ruijie Shi, Weize Chen, Cheng Yang, Xiaoyin Che, Ye Tian, Xuantang Xiong, and Lei Han. Multi-agent collaboration via evolving orchestration. 2025. Preprint.

[92] Changhua Pei, Zexin Wang, Fengrui Liu, Zeyan Li, Yang Liu, Xiao He, Rong Kang, Tieying Zhang, Jianjun Chen, Jianhui Li, Gaogang Xie, and Dan Pei. Flow-of-action: Sop enhanced llm-based multi-agent system for root cause analysis. 2025. Preprint.

[93] Qian Chen, Zihao Xie, Yifei Wang, Wei Liu, Kunlun Zhu, Hanchen Xia, Yufan Dang, Z. Z. Du, W. Chen, Cheng Yang, Zhiyuan Liu, and Maosong Sun. Scaling large language model-based multi-agent collaboration. arXiv (Cornell University), 2024.

[94] Tianjun Zhang, Huazhe Xu, Xiaolong Wang, Yi Wu, Kurt Keutzer, Joseph E. Gonzalez, and Yuandong Tian. Multi-agent collaboration via reward attribution decomposition. arXiv (Cornell University), 2020.

[95] Hong Cao, Rong Ma, Yanlong Zhai, and Jun Shen. Llm-collab: a framework for enhancing task planning via chain-of-thought and multi-agent collaboration. Applied Computing and Intelligence, 2024.

[96] Yongchao Chen, Jacob Arkin, Yang Zhang, Nicholas Roy, and Chuchu Fan. Scalable multi-robot collaboration with large language models: Centralized or decentralized systems? 2024 IEEE International Conference on Robotics and Automation (ICRA), 2024.

[97] Zhuoyun Du, Chen Qian, Wei Liu, Zihao Xie, Yifei Wang, Rennai Qiu, Yufan Dang, Weize Chen, Cheng Hong Yang, Ye Tian, Xuantang Xiong, and Lei Han. Multi-agent collaboration via cross-team orchestration. arXiv (Cornell University), 2024.

[98] Ziqi Jia, Junjie Li, Xiaoyang Qu, and Jianzong Wang. Enhancing multi-agent systems via reinforcement learning with llm-based planner and graph-based policy. 2025 IEEE International Conference on Robotics and Automation (ICRA), 2025.

[99] Pengfei He, Yuping Lin, Shen Dong, Han Xu, Yue Xing, and Hui Liu. Red-teaming llm multi-agent systems via communication attacks. 2025. Preprint.

[100] Yongjian Guo, Puzhuo Liu, Wanlun Ma, Zehang Deng, Xiaogang Zhu, Peng Di, Xi Xiao, and Sheng Wen. Mcpxkit: The unified toolkit for analyzing model context protocol security. IEEE Transactions on Dependable and Secure Computing, 2026.

[101] Shi Liu, Xuehai Tang, Xikang Yang, Liang Lin, Biyu Zhou, Wenjie Xiao, and Wantao Liu. When the manual lies: A realistic benchmark to evaluate mcp poisoning attacks for llm agents. 2026 29th International Conference on Computer Supported Cooperative Work in Design (CSCWD), 2026.

[102] Kaibo Huang, Yukun K Wei, WanshengWu, Tianhua Zhang, Zhongliang Yang, and Linna Zhou. Whispering agents: An event-driven covert communication protocol for the internet of agents. Proceedings of the AAAI Conference on Artificial Intelligence, 2026.

[103] Bo Zhao, Konda Reddy Mopuri, and Hakan Bilen. idlg: Improved deep leakage from gradients. arXiv (OAI-PMH), 2020.

[104] Satsuya Ohata and Koji Nuida. Communication-efficient (client-aided) secure two-party protocols and its application. arXiv (OAI-PMH), 2020.

[105] Yen-Cheng Liu, Junjiao Tian, Chih-Yao Ma, Nathan Glaser, Chia-Wen Kuo, and Zsolt Kira. Who2com: Collaborative perception via learnable handshake communication. 2020. Preprint.